



Python Schnelleinstieg

Michael Inden

Freiberuflicher Trainer und Autor



<https://github.com/Michaeli71/Python-Workshop>

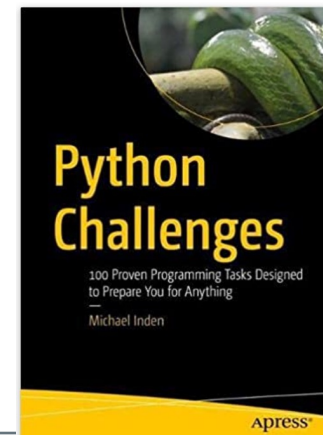
Speaker Intro



- **Michael Inden**, Jahrgang 1971, Dipl.-Inform., Uni Oldenburg
- ~8 ¼ Jahre **SSE** bei Heidelberger Druckmaschinen AG in Kiel
- ~6 ¾ Jahre **TPL, SA** bei IVU Traffic Technologies AG in Aachen
- ~4 ¼ Jahre **LSA / Trainer** bei Zühlke Engineering AG in Zürich
- ~3 Jahre **TL / CTO** bei Direct Mail Informatics / ASMIQ in Zürich
- ~3 ½ Jahre **Head of Development** bei Adcubum in Zürich
- Ab 1.4. **Dozent** für Software Engineering an der OST Rapperswil
- Freiberuflicher **Consultant, Trainer** und **Konferenz-Speaker**
- Autor und Gutachter bei dpunkt.verlag, O'Reilly und APress

E-Mail: michael_inden@hotmail.com

Kurse: **Bitte spricht mich an!**



<https://github.com/Michaeli71/Python-Workshop>



Agenda



- **PART 1: Einstieg Python**

- Einführung & Trend
- Syntax im Kurzüberblick
- Funktionen definieren
- Module ausführbar machen / `main()`

- **PART 2: Strings**

- Wichtige Funktionen / Methoden
- Slicing
- Formatierte Ausgaben
- Mehrzeilige Strings
- Pytest Quick Start





- **PART 3: Klassen & Objektorientierung**
 - Basics
 - Textuelle Ausgaben
 - Gleichheit `is` / `==` und `==` / `equals()`
 - Unterschiede zu Java / Information Hiding
 - Vererbung und Typprüfung
 - Enums
 - Named Tuple / Java Record





- **PART 4: Collections**

- Schnelleinstieg `list`
 - Comprehensions
 - Slicing – Zugriff auf Teilbereiche
- Schnelleinstieg `set`
- Schnelleinstieg `dict`

- **PART 5: Collections Advanced**

- Sortierung – `sort()` / `sorted()` + Analogie Comparator





- **PART 6: Exception-Handling**
 - Schnelleinstieg
 - Exceptions selbst auslösen
 - Eigene Exception-Typen definieren
 - Propagation von Exceptions
- **PART 7: Dateiverarbeitung**
 - Verzeichnisse und Dateien verwalten
 - Daten schreiben / lesen
 - JSON verarbeiten





- **PART 8: Datumsverarbeitung**
 - Einführung Datumsverarbeitung
 - Zeitpunkte und die Klasse `datetime`
 - Datumswerte und die Klasse `date`
 - Zeit und die Klasse `time`
- ✂️💻⚙️ **Python Quick Start** ⚙️💻✂️: **Projects & Productivity Showdown**





- **PART 99: BONUS**
 - «Reflection»
 - «HTTP-Support»
 - «PyTest»
 - «QR Code»





PART 1:

Einstieg Python

- Einführung & Trend
 - Funktionen definieren
 - Module ausführbar machen / `main()`
-



- **Anfang der 1990er Jahre von Guido van Rossum als Skriptsprache entwickelt**
 - **1994 in Version 1.0 veröffentlicht.**
 - **Mittlerweile hat Python zwar auch schon mehr als 25 Jahre auf dem Buckel, wird aber nicht altersschwach, sondern kontinuierlich gepflegt und weiterentwickelt.**
 - **Python 3 erschien bereits 2008 und hat diverse Aktualisierungen erfahren, derzeit Version 3.14 aktuell (im Oktober 2025 erschienen)**
 - **Für Oktober 2026 ist Python 3.15 geplant**
-

Aktueller Stand (März 26 / März 25 unten)



Mar 2026	Mar 2025	Change	Programming Language		Ratings	Change
1	1			Python	21.25%	-2.59%
2	4	▲		C	11.55%	+2.02%
3	2	▼		C++	8.18%	-2.90%
4	3	▼		Java	7.99%	-2.37%
5	5			C#	6.36%	+1.49%

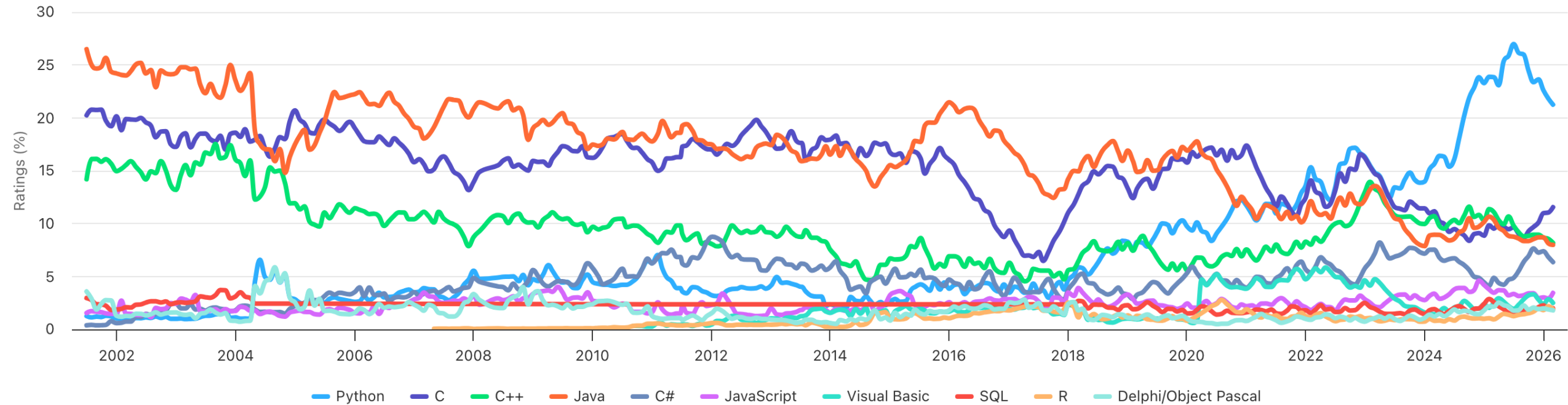
Feb 2025	Feb 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.88%	+8.72%
2	3	▲		C++	11.37%	+0.84%
3	4	▲		Java	10.66%	+1.79%
4	2	▼		C	9.84%	-1.14%
5	5			C#	4.12%	-3.41%

Popularitätstrends der Top 10 Programmiersprachen (März 26)



TIOBE Programming Community Index

Source: www.tiobe.com



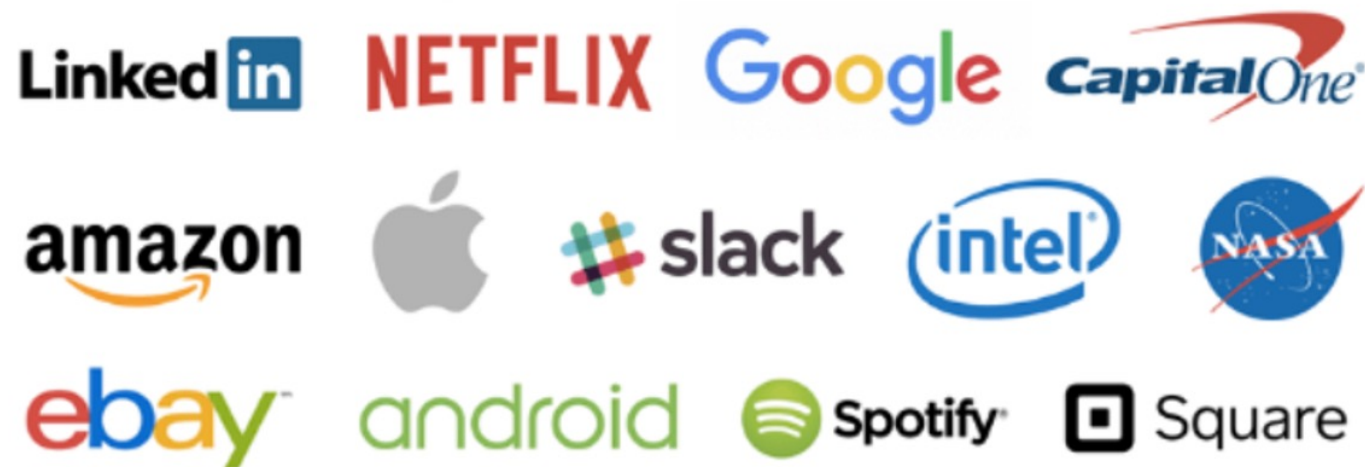
Vergleich Python – Java



- Wer nutzt Python?



- Wer nutzt Java?



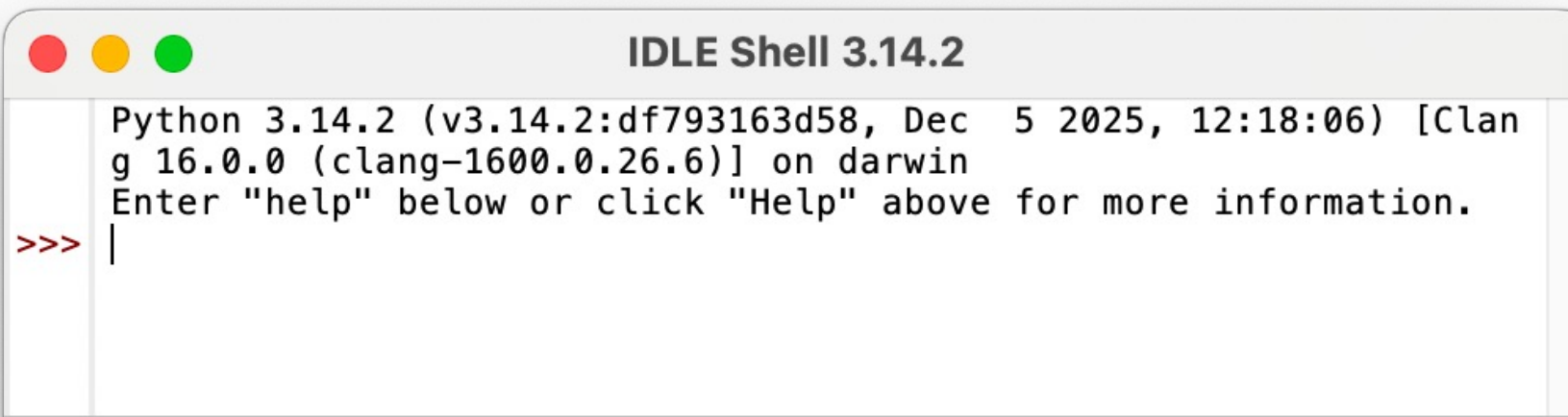


- **Python**

```
$ python3
Python 3.14.2 (v3.14.2:df793163d58, Dec 5 2025, 12:18:06) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

- **IDLE**

>  Python 3.13	18 Aug 2025 at 11:30
✓  Python 3.14	6 Dec 2025 at 17:38
 IDLE	6 Dec 2025 at 17:38
 Install Certificates.command	5 Dec 2025 at 18:38



```
Python 3.14.2 (v3.14.2:df793163d58, Dec 5 2025, 12:18:06) [Clan
g 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>> |
```

Ln: 3 Col: 0



- **str** – Textuelle Informationen, wie z. B. "Moin" oder 'Maria'. String-Werte sind von doppelten oder einfachen Anführungszeichen eingeschlossen
- **int** – Ganzzahlen wie 123, oder -4711
- **float** – Gleitkommazahlen, als mit Vor- und Nachkommastellen, wie 72.71 oder -1.357
- **bool** – Wahrheitswerte als wahr oder falsch, in Python: True oder False.

```
>>> type("Python")
<class 'str'>
>>> type("1234")
<class 'str'>
>>> type(1234)
<class 'int'>
>>> type(42.195)
<class 'float'>
>>> type(True)
<class 'bool'>
```




- **Mathematische Operatoren**

```
>>> 5 + 4 - 2  
7
```

```
>>> 7 * 2  
14
```

```
>>> 14 / 3  
4.666666666666667
```

```
>>> 14 // 3  
4
```

```
>>> 14 % 3  
2
```

```
>>> 2 ** 8  
256
```



- **Minimum aus 3 Werten**

```
def min_of_3(x, y, z):  
    if x < y:  
        if x < z:  
            return x  
        else:  
            return z  
    else:  
        if y < z:  
            return y  
        else:  
            return z
```

- **Basierend auf Built-in-Funktion min()**

```
def min_of_3(x, y, z):  
    return min(x, min(y, z))
```

Besonderheit: Verschachtelte Funktionen definieren



```
def f():  
    print("f1")  
  
    def g():  
        print("g!")  
  
        # (noch) nicht sichtbar!  
        # h()  
        def h():  
            print("h!")  
  
        h()  
  
        # nicht (mehr) sichtbar!  
        # h()  
        g()  
        print("f2")  
  
f()  
print("nach f")
```

Besonderheit: Verschachtelte Funktionen definieren



```
def f():  
    print("f1")  
  
    def g():  
        print("g!")  
  
        # (noch) nicht sichtbar!  
        # h()  
        def h():  
            print("h!")  
  
            h()  
  
        # nicht (mehr) sichtbar!  
        # h()  
        g()  
        print("f2")  
  
f()  
print("nach f")
```

```
f1  
g!  
h!  
f2  
nach f
```



- **Gegeben sei folgende Funktion**

```
def parameter_example(first, second):  
    print("first:", first)  
    print("second:", second)
```

- **Bekanntermaßen und wie gewohnt lässt sich die Funktion wie folgt aufrufen:**

```
parameter_example(1, 22)
```

- **Als Variation ermöglicht Python eine Parameterübergabe per Name:**

```
parameter_example(second = 22, first = 1)
```

- **Beides Mal mit dem gleichen Resultat:**

```
first: 1  
second: 22
```



- Mitunter ist es hilfreich, einen Defaultwert für einen Parameter bereitstellen zu können

```
def parameter_with_default(x, y, opt_info="n/a"):
    print("(%d, %d)" % (x, y))
    print("Info:", opt_info)
```

```
parameter_with_default(7, 2)
parameter_with_default(72, 71, "OPT-VALUE")
```

```
(7, 2)
Info: n/a
(72, 71)
Info: OPT-VALUE
```



- Mitunter ist es hilfreich, eine beliebige Anzahl an Werten an eine Funktion zu übergeben
- Dazu wird vor dem Parameter ein Stern, etwa `*args` angegeben
- Tatsächlich kennen wir bereits eine Funktion, die eine solche variable Anzahl an Argumenten unterstützt! Mehrmals haben wir nun schon die Funktion `print()` mit einer kommaseparierten Aufzählung von Werten aufgerufen.

```
def flexi_print(*values):  
    print(*values)  
    print(values)
```

```
flexi_print("ONE")  
flexi_print("ONE", "TWO")  
flexi_print("ONE", "TWO", "THREE")
```



- ***values** => weiterreichen als einzelne Parameter
- **values** => weiterreichen als **Tupel**

```
def flexi_print(*values):  
    print(*values)  
    print(values)
```

```
flexi_print("ONE")  
flexi_print("ONE", "TWO")  
flexi_print("ONE", "TWO", "THREE")
```

```
ONE  
( 'ONE', )  
ONE TWO  
( 'ONE', 'TWO' )  
ONE TWO THREE  
( 'ONE', 'TWO', 'THREE' )
```




- **Listen**

```
>>> names = ["Tim", "Michael", "Tom", "Mike", "Michael"]
>>> names.append("John")
>>> names += ["Jim", "James"]
>>> names
['Tim', 'Michael', 'Tom', 'Mike', 'Michael', 'John', 'Jim', 'James']
```

- **Zugriff & Abfrage**

```
>>> names[1]
'Michael'
>>> names.index("Tom")
2
>>> "Tom" in names
True
```



- **Tupel**

```
>>> pizza_top_3 = ("diavolo", "napoli", "funghi")
>>>
>>> pizza_top_3[0]
'diavolo'
>>> pizza_top_3.index("diavolo")
0
```

- **Sets**

```
>>> fruits = {"Apple", "Ananas"}
>>> fruits.add("Apple")
>>> fruits.add("Bananas")
>>> fruits
{'Apple', 'Bananas', 'Ananas'}

>>> "Apple" in fruits
True
```



- **Dictionaries (Maps)**

```
>>> city_inhabitants = {"Kiel": 250_000,  
...                     "Bremen": 550_000,  
...                     "Zürich": 400_000}  
>>> city_inhabitants  
{'Kiel': 250000, 'Bremen': 550000, 'Zürich': 400000}  
  
>>> city_inhabitants.keys()  
dict_keys(['Kiel', 'Bremen', 'Zürich'])  
  
>>> city_inhabitants.values()  
dict_values([250000, 550000, 400000])  
  
>>> city_inhabitants.items()  
dict_items([('Kiel', 250000), ('Bremen', 550000), ('Zürich', 400000)])
```



- **Dictionaries (Maps)**

```
>>> city_inhabitants.get("Zürich")  
400000
```

```
>>> city_inhabitants["Zürich"]  
400000
```

```
>>> city_inhabitants.get("Aachen")  
>>> print(city_inhabitants.get("Aachen"))  
None
```

```
>>> city_inhabitants["Aachen"]  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 'Aachen'
```



- **Wertebereiche**

```
>>> range(1, 10)
range(1, 10)
```

```
>>> list(range(1, 10))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- **Wertebereiche mit Schrittweite / auch negativ**

```
>>> list(range(2, 21, 3))
[2, 5, 8, 11, 14, 17, 20]
```

```
>>> list(range(21, 2, -3))
[21, 18, 15, 12, 9, 6, 3]
```



- **Indexbasierte for-in-Schleife**

```
>>> for i in range(2, 5):  
...     print("Durchlauf:", i)  
...  
Durchlauf: 2  
Durchlauf: 3  
Durchlauf: 4
```

- **Indexbasierte for-in-Schleife**

```
>>> cities = ['Kiel', "Bremen", 'Zürich', 'Basel']  
>>> for i in range(len(cities)):  
...     print(cities[i])  
...  
Kiel  
Bremen  
Zürich  
Basel
```



- **For-in-VALUES**

```
>>> for current_name in ["Barbara", "Lilija", "Sophie"]:  
...     print(current_name)  
...  
Barbara  
Lilija  
Sophie
```

- **For-In-REVERSED**

```
>>> for current_name in reversed(["Barbara", "Lilija", "Sophie"]):  
...     print(current_name)  
...  
Sophie  
Lilija  
Barbara
```



- **For-in-SORTED**

```
>>> for value in sorted([7, 2, 3, 5, 11, 17, 19, 13]):  
...     print(value)  
...  
2  
3  
5  
7  
11  
13  
17  
19
```




- Zugriff auf Index und Wert ohne enumerate()

```
>>> message = ["Python", "has", "several", "loop", "variants"]
>>> for i in range(len(message)):
...     print(i, message[i], end=',')
...
0 Python,1 has,2 several,3 loop,4 variants,
```

- Mit enumerate() – liefert Tupel

Ergebnis ist ein Tupel
aus Index und Wert

```
>>> message = ["Python", "has", "several", "loop", "variants"]
>>> for i, current_word in enumerate(message):
...     print(i, current_word, end=',')
...
0 Python,1 has,2 several,3 loop,4 variants,
```



- **Kombination von enumerate() und sorted()**

```
>>> for idx, value in enumerate(sorted([7, 2, 3, 5, 11, 17, 19, 13])):  
...     print(idx, value)  
...  
0 2  
1 3  
2 5  
3 7  
4 11  
5 13  
6 17  
7 19
```



- **Kombination von `enumerate()` und `reversed()` und `sorted()`**

```
>>> for idx, value in enumerate(reversed(sorted([7, 2, 3, 5, 11, 17, 19, 13]))):  
...     print(idx, value)  
...  
0 19  
1 17  
2 13  
3 11  
4 7  
5 5  
6 3  
7 2
```



- **while <cond>:**

```
>>> i = 0
>>> while i < 5:
...     print(i)
...     i += 1
...
0
1
2
3
4
```

ABER: KEIN DO-WHILE in Python!



- Zufallsfunktionalität aus dem Modul random

```
>>> import random
```

```
>>> random.randrange(2, 27)
```

```
24
```

```
>>> random.randrange(2, 27)
```

```
12
```

```
>>> random.choice(["Pizza", "Pasta", "Steak", "Salat"])
```

```
'Steak'
```

```
>>> random.choice(["Pizza", "Pasta", "Steak" ", "Salat"])
```

```
'Pizza'
```



**Woher wussten wir, was
man aufrufen kann?**



- Übersicht, was das Modul random bietet:

```
>>> dir(random)
['BPF', 'LOG4', 'NV_MAGICCONST', 'RECIP_BPF', 'Random', 'SG_MAGICCONST',
'SystemRandom', 'TWOPI', '_ONE', '_Sequence', '__all__', '__builtins__',
'__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__',
'__spec__', '_accumulate', '_acos', '_bisect', '_ceil', '_cos', '_e', '_exp',
'_fabs', '_floor', '_index', '_inst', '_isfinite', '_lgamma', '_log', '_log2',
'_os', '_parse_args', '_pi', '_random', '_repeat', '_sha512', '_sin', '_sqrt',
'_test', '_test_generator', '_urandom', 'betavariate', 'binomialvariate',
'choice', 'choices', 'expovariate', 'gammavariate', 'gauss', 'getrandbits',
'getstate', 'lognormvariate', 'main', 'normalvariate', 'paretovariate',
'randbytes', 'randint', 'random', 'randrange', 'sample', 'seed', 'setstate',
'shuffle', 'triangular', 'uniform', 'vonmisesvariate', 'weibullvariate']
```

Module entdecken ... Built-in help()



```
>>> help(random)
```

```
michaelinden — less Python — 77x35
Help on module random:

NAME
    random — Random variable generators.

MODULE REFERENCE
    https://docs.python.org/3.14/library/random.html

    The following documentation is automatically generated from the Python
    source files. It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations. When in doubt, consult the module reference at the
    location listed above.

DESCRIPTION
    bytes
    -----
        uniform bytes (values between 0 and 255)

    integers
    -----
        uniform within range

    sequences
    -----
        pick random element
        pick random sample
        pick weighted random sample
        generate random permutation

    distributions on the real line:
    -----
        uniform
        triangular

Help on random line 1 (press h for help or q to quit)
```

random — Generate pseudo-random numbers

Source code: [Lib/random.py](#)

This module implements pseudo-random number generators for various distributions.

For integers, there is uniform selection from a range. For sequences, there is uniform selection of a random element, a function to generate a random permutation of a list in-place, and a function for random sampling without replacement.

On the real line, there are functions to compute uniform, normal (Gaussian), lognormal, negative exponential, gamma, and beta distributions. For generating distributions of angles, the von Mises distribution is available.

Almost all module functions depend on the basic function [random\(\)](#), which generates a random float uniformly in the half-open range `0.0 <= X < 1.0`. Python uses the Mersenne Twister as the core generator. It produces 53-bit precision floats and has a period of $2^{19937}-1$. The underlying implementation in C is both fast and threadsafe. The Mersenne Twister is one of the most extensively tested random number generators in existence. However, being completely deterministic, it is not suitable for all purposes, and is completely unsuitable for cryptographic purposes.

The functions supplied by this module are actually bound methods of a hidden instance of the [random.Random](#) class. You can instantiate your own instances of [Random](#) to get generators that don't share state.

Class [Random](#) can also be subclassed if you want to use a different basic generator of your own devising: see the documentation on that class for more details.

The [random](#) module also provides the [SystemRandom](#) class which uses the system function [os.urandom\(\)](#) to generate random numbers from sources provided by the operating system.

Warning: The pseudo-random generators of this module should not be used for security purposes. For



- **Module sind in Python lediglich Dateien mit der Endung .py und enthalten etwas Python-Code**
- **Problem: Beim import wird der Code des importierten Moduls immer Zeile für Zeile abgearbeitet / ausgeführt**
- **Abhilfe: Funktionalität in Funktionen, insbesondere dedizierte main() auslagern**

```
def main():  
    # Hauptprogramm
```

```
if __name__ == "__main__":  
    main()
```



DEMO

`own_module.py`



Exercises Part 1

<https://github.com/Michaeli71/Python-Workshop>





PART 2:

Strings

- Wichtige Funktionen / Methoden
 - Slicing
 - Formatierte Ausgaben
 - Mehrzeilige Strings
 - Pytest Quick Start
-

Strings



- **Definition**

```
str1 = "DOUBLE QUOTED STRING"  
str2 = 'SINGLE QUOTED STRING'
```

- **Vorteil: Mix möglich!**

```
>>> str1 = "DOUBLE 'contains' single"  
>>> str2 = 'SINGLE "contains" double'  
>>> str1  
"DOUBLE 'contains' single"  
>>> str2  
'SINGLE "contains" double'
```



- **Unicode (\u)**

```
>>> str_unicode = "\u1234 \u2620 \u2744 \u24C2"  
>>> str_unicode  
'ǀ ☠️ ❄️ Ⓜ'
```

- **Unicode mit Surrogates (\U)**

```
>>> str_unicode = "\U0001F601 \U0001F631"  
>>> str_unicode  
'😄 😱'
```

- **Unicode mit Namen (\N{name})**

```
>>> str_unicode = "\N{dog}\N{cat}\N{bird}\N{mouse}\N{snowman}"  
>>> str_unicode  
'🐶 🐱 🐤 🐭 🌨️'
```



- **Groß- und Kleinschreibung**

```
>>> message = "IMPORTANT: Please consult the doctor"
>>> message.upper()
'IMPORTANT: PLEASE CONSULT THE DOCTOR'
>>> message.lower()
'important: please consult the doctor'
```

- **Unveränderlichkeit!!!**

```
>>> message
'IMPORTANT: Please consult the doctor'
```

LOWER => CASEFOLD

```
>>> "Fluß".lower()
'fluß'
>>> "Fluß".casefold()
'fluss'
```

```
>>> "Straße".upper()
'STRASSE'
>>> "Straße".lower()
'straße'
>>> "Straße".casefold()
'strasse'
```



- **Konkatenation (+)**

```
>>> last_name = "Inden"
>>> "Michael" + " " + last_name
'Michael Inden'
```

- **ABER:**

```
>>> "Bitte " + 2 + " mal klingeln"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

- **Lösung mit Cast:**

```
>>> "Bitte " + str(2) + " mal klingeln"
'Bitte 2 mal klingeln'
```

Strings



- **center()**

```
>>> value = "This text has blanks at the beginning and the end"
>>> value_with_blanks = value.center(55)
>>> value_with_blanks
'   This text has blanks at the beginning and the end   '
```

- **strip()**

```
>>> print("strip():  '", value_with_blanks.strip(), "'", sep="")
strip(): 'This text has blanks at the beginning and the end'
```

- **lstrip() / rstrip():**

```
>>> print("lstrip():  '", value_with_blanks.lstrip(), "'", sep="")
lstrip(): 'This text has blanks at the beginning and the end '
```

```
>>> print("rstrip():  '", value_with_blanks.rstrip(), "'", sep="")
rstrip(): '   This text has blanks at the beginning and the end'
```

Strings



- **Länge ermitteln – len()**

```
>>> content = "This is a short message"
>>> len(content)
23
```

- **Leerstring?**

```
>>> no_content = ""
>>> len(no_content) == 0
True
```

- **Prüfungsvariante (eher «pythonic»)**

```
>>> if not no_content:
...     print("empty")
...
empty
```



- **Alles Zeichen oder alles Ziffern?**

```
>>> pincode = "1234"
>>> pincode.isalpha()
False
>>> pincode.isdigit()
True
>>> pincode = "A4711"
>>> pincode.isalpha()
False
>>> pincode.isdigit()
False
>>> pincode = "ABC"
>>> pincode.isalpha()
True
```



- **Auf Zeichen per Index zugreifen**

```
>>> content = "This is a short message"
>>> content[0]
'T'
>>> content[3]
's'
```

- **Besonderheit: Negative Indizes möglich ($-idx \Leftrightarrow \text{len}(\text{txt}) - idx$)**

```
>>> content = "This is a short message"
>>> content[-1]
'e'
>>> content[-2]
'g'
>>> content[-3]
'a'
>>> content[-20]
's'
```



```
>>> content[len(content) - 1]
'e'
>>> content[len(content) - 2]
'g'
>>> content[len(content) - 3]
'a'
>>> content[len(content) - 20]
's'
```



- **Bereichsverletzung**

```
>>> content = "This is a short message"
>>> content[25]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

- **Bereichsverletzung bei negativem Index**

```
>>> content = "This is a short message"
>>> content[-25]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```



- **String positionsbasiert durchlaufen**

```
message = "Python has several loop variants"
for i in range(len(message)):
    print(i, message[i], end=',')
```

- **Zeichenbasiert**

```
for current_char in message:
    print(current_char, end=',')
```

- **Kombination: Mit Position und aktuellem Zeichen (=> Vorteil Built-in-Tupel)**

```
for i, current_char in enumerate(message):
    print(i, current_char, end=',')
```

Strings



- **String positionsbasiert durchlaufen**

```
message = "Python has several loop variants"
for i in range(len(message)):
    print(i, message[i], end=', ')
```

- **Zeichenbasiert**

```
for current_char in message:
    print(current_char, end=', ')
```

P,y,t,h,o,n, ,h,a,s, ,s,e,v,e,r,a,l,
,l,o,o,p, ,v,a,r,i,a,n,t,s,

- **Kombination: Mit Position und aktuellem Zeichen**

```
for i, current_char in enumerate(message):
    print(i, current_char, end=', ')
```

0 P,1 y,2 t,3 h,4 o,5 n,6 ,7 h,8 a,9 s,10 ,11 s,12 e,13 v,14 e,15 r,16 a,17
l,18 ,19 l,20 o,21 o,22 p,23 ,24 v,25 a,26 r,27 i,28 a,29 n,30 t,31 s,

Strings



- **Slicing => Extraktion eines neuen Strings von Position startidx bis endidx**

`text[startidx:endidx]`

- **Beispiel**

```
teile = "Dies ist ein String. Rest ABC"
```

```
print(teile[0:4])
```

```
print(teile[5:8])
```

```
print(teile[9:12])
```

```
print(teile[9:19])
```

```
print(teile[19:])
```



```
Dies  
ist  
ein  
ein String  
. Rest ABC
```

- **Index negativ**

```
>>> print(teile[-3:-1])
```

```
AB
```

```
>>> print(teile[-3:])
```

```
ABC
```





- Slicing – Besonderheiten

`text[startidx:endidx:step]`

- Beispiel

```
teile = "Dies ist ein String. Rest ABC"
```

```
# Besonderheiten
```

```
print(teile[:])
```

Dies ist ein String. Rest ABC

```
print(teile[::-1])
```

```
print(teile[19::-1])
```

```
print(teile[:8:-1])
```

```
print(teile[-5:8:-1])
```



CBA tseR .gnirtS nie tsi seiD

.gnirtS nie tsi seiD

CBA tseR .gnirtS nie

tseR .gnirtS nie



- **Enthaltensein (in)**

```
>>> msg = "Tim arbeitet in Kiel. Michael lebt in Zürich"
```

```
>>> "Michael" in msg  
True
```

```
>>> "arbeitet" in msg  
True
```

```
>>> "Bremen" in msg  
False
```



- **Textbestandteil am Anfang und Ende prüfen**

```
>>> msg = "Important Info"
>>> msg.startswith("Impo")
True
>>> msg.endswith("Info")
True
```

- **Vorkommen zählen**

```
>>> msg = "tri tra tru lala"
>>> msg.count("tr")
3
>>> msg.count("tra")
1
>>> msg.count("la")
2
>>> msg.count("a")
3
```



- **Suchen und Weitersuchen**

```
>>> msg = "This is a tiny story. This tiny text ends now."
>>> msg.find("tiny")
10
>>> msg.rfind("tiny")
27
>>> msg.index("tiny")
10
>>> msg.rindex("tiny")
27
>>> msg.find("tiny", 11)
27
>>> msg.find("Michael")
-1
>>> msg.index("Michael")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
```





- **Strings wiederholen (*)**

```
>>> greeting = "MOIN"  
>>> greeting * 2  
'MOINMOIN'
```

```
>>> nonsens = "BLA"  
>>> nonsens * 3  
'BLABLABLA'
```

- **Spezialschreibweise**

```
>>> msg = "my name is michael"  
>>> msg.capitalize()  
'My name is michael'  
>>> msg.title()  
'My Name Is Michael'  
>>> 'My Name Is Michael'.swapcase()  
'mY nAME iS mICHAEL'
```



- **Strings aufspalten**

```
>>> timestamp = "11:22:33"  
>>> timestamp.split(":")  
['11', '22', '33']
```

- **Strings aufspalten / zusammenführen und ersetzen**

```
>>> msg = "One Two Three Four"  
>>> msg.split(" ")  
['One', 'Two', 'Three', 'Four']
```

```
>>> "-".join(['One', 'Two', 'Three', 'Four'])  
'One-Two-Three-Four'
```

```
>>> 'One-Two-Three-Four'.replace("-", "...")  
'One...Two...Three...Four'
```



- **Verschiedene Varianten**

```
product = "Apple iMac"  
price = 3699
```

```
# Varianten der formatierten Ausgabe  
print("the", product, "costs", price)  
print("the {} costs {}".format(product, price))  
print("the %s costs %d" % (product, price))  
print(f"the {product} costs {price}")
```

- **Führen zu folgenden Ausgaben**

```
the Apple iMac costs 3699  
the Apple iMac costs 3699  
the Apple iMac costs 3699  
the Apple iMac costs 3699
```



- **Spezielle Zeichen im String (hier ungewünscht Newline / Tabs)**

```
>>> path = "C:\newcontent\tim\news"  
>>> path  
'C:\newcontent\tim\news'  
>>> print(path)  
C:  
ewcontent im  
ews
```

- **Raw-Strings (r"") als Abhilfe**

```
>>> path = r"C:\newcontent\tim\news"  
>>> path  
'C:\\newcontent\\tim\\news'  
>>> print(path)  
C:\newcontent\tim\news
```




- **Mehrzeilige Strings (""")**

```
>>> multi_line_string = """Line 1
...     Line 2
...     Line 3"""
>>> multi_line_string
'Line 1\n    Line 2\n    Line 3'
```

- **Mehrzeilige Strings können einfache Anführungszeichen enthalten**

```
>>> multi_line_string_with_quotes = """Line 1
... the "second" line contains 'quotes'
... last line"""
>>>
>>> multi_line_string_with_quotes
'Line 1\nthe "second" line contains \'quotes\'\nlast line'
```



- **Mehrzeilige Strings mit Platzhaltern**

```
>>> value_dict = {"info" : "text", "costs" : "for free",
...               "action" : "consult", "document" : "help"}
>>>
>>> multi_line_string_with_dict = """This {info} is {costs}.
... Please {action} the {document}""".format_map(value_dict)
>>>
>>> multi_line_string_with_dict
'This text is for free.\nPlease consult the help'
```

- **Mehrzeilige Strings sind normale Strings**

```
>>> type(multi_line_string_with_dict)
<class 'str'>
>>> len(multi_line_string_with_dict)
46
```

Strings => Docstrings



- **Mehrzeilige Strings zur Dokumentation**

```
def greet(name):  
    """Returns a greeting message with the provided name."""  
    return f"Hello, {name}!"
```

```
def multiply(a: int, b: int) -> int:  
    """  
    Multiplies two integers and returns the result.  
  
    Args:  
        a (int): The first integer.  
        b (int): The second integer.  
  
    Returns:  
        int: The product of a and b.  
    """  
    return a * b
```

```
help(greet)
```

```
help(multiply)
```

Help on function greet in module __main__:

```
greet(name)
```

Returns a greeting message with the provided name.

Help on function multiply in module __main__:

```
multiply(a: int, b: int) -> int
```

Multiplies two integers and returns the result.

Args:

a (int): The first integer.

b (int): The second integer.

Returns:

int: The product of a and b.



**Wie kann ich
Werte einlesen?**



- **Von Konsole einlesen => Achtung: die Rückgabe ist vom Typ str**

```
>>> input("What's your name?")
What's your name? Michael
'Michael'
```

```
>>> input("How old are you? ")
How old are you? 52
'52'
```

- **Als Ganzzahl einlesen**

```
>>> age = int(input("How old are you? "))
How old are you? 52
>>> age
52
```



- **eval()** erlaubt, beliebige, stringbasierte Python-Ausdrücke auszuwerten:

```
>>> eval("7 * 2")
14
>>> eval("print('Hello Michael')")
Hello Michael
>>> x = 1_000
>>> eval("x ** 2")
1000000
```

- **Aber keine Folge von Anweisungen:**

```
>>> eval("if x > 10: print('higher than 10')")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    File "<string>", line 1
      if x > 10: print('higher than 10')
      ^^
SyntaxError: invalid syntax
```



«Pytest Quick Start»





- Für Tests sofort (nach Installation) einsetzbar (<https://docs.pytest.org/en/stable/>)
- Testbehauptungen werden einfach mit dem Schlüsselwort `assert` formuliert

```
def test_answer():  
    assert sum([1, 2, 3]) == 6
```

- Auf einfache Weise auch parametrisierte Tests möglich

```
import pytest  
  
@pytest.mark.parametrize("test_input,expected",  
                          [ ("2 + 5", 7), ("2 ** 4", 16), ("6 * 9", 42)])  
def test_eval(test_input, expected):  
    assert eval(test_input) == expected
```

Pytest Quick Start



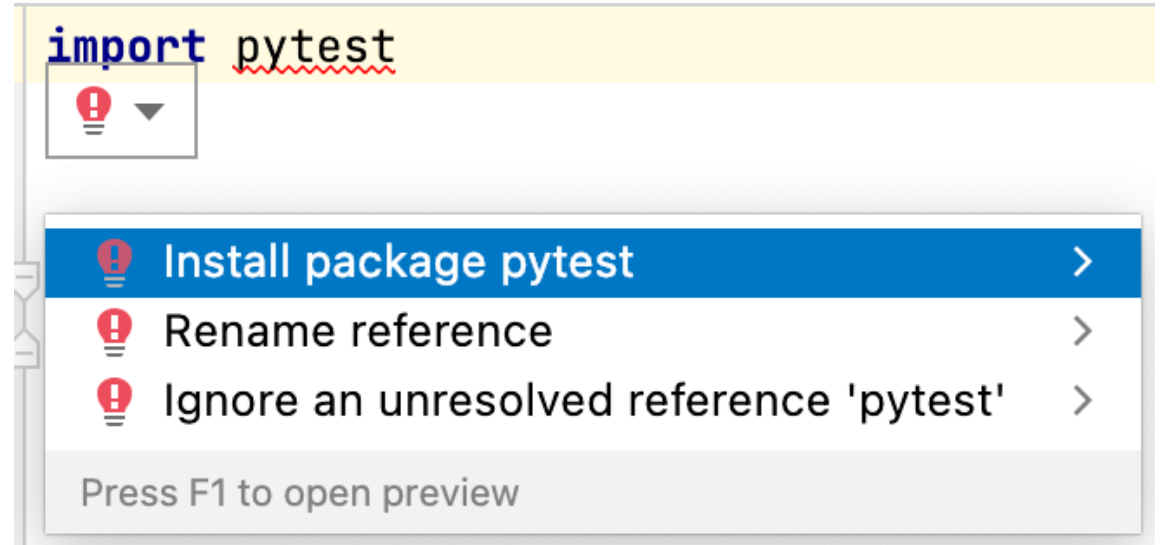
- Gute Integration in PyCharm
- Tests beginnen mit «test_»

```
import pytest
```

```
def not_detected_test():  
    assert sum([1, 2, 3]) == 6
```

```
def test_answer():  
    assert sum([1, 2, 3]) == 6
```

```
@pytest.mark.parametrize("test_input,expected",  
                           [ ("2 + 5", 7), ("2 ** 4", 16), ("6 * 9", 42)])  
def test_eval(test_input, expected):  
    assert eval(test_input) == expected
```



Pytest Quick Start



- Gute Integration in PyCharm
- Tests beginnen mit «test_»

```
import pytest
```

```
def not_detected_test():  
    assert sum([1, 2, 3]) == 6
```

```
def test_answer():  
    assert sum([1, 2, 3]) == 6
```

```
@pytest.mark.parametrize("test_input,expected",  
                           [ ("2 + 5", 7), ("2 ** 4", 16), ("6 * 9", 42)])
```

```
def test_eval(test_input, expected):  
    assert eval(test_input) == expected
```

Test Results	0 ms
pytest_intro	0 ms
test_answer	0 ms
test_eval	0 ms
(2 + 5-7)	0 ms
(2 ** 4-16)	0 ms
(6 * 9-42)	0 ms



DEMO

`pytest_intro_test.py`



Exercises Part 2

<https://github.com/Michaeli71/Python-Workshop>





PART 3:

Klassen

- Basics
 - Textuelle Ausgaben
 - Gleichheit `is` / `==` und `==` / `equals()`
 - Unterschiede zu Java / Information Hiding
 - Vererbung und Typprüfung
 - Enums
 - Named Tuple / Java Record
-

Klassen definieren



- Spezielle Methode `__init__()` ist der Konstruktor
- Attribute werden einfach im Konstruktor aufgeführt
- `self` ist wie `this`, `None` ist wie `null`
- Keine wirklichen Sichtbarkeiten
- Attribute sind standardmässig «public»

```
class Car:
    def __init__(self):
        self.brand = None
        self.color = None
        self.horse_power = 0
```

- Objektkonstruktion einfach durch Aufruf (**ohne** `new`)

```
car = Car()
```



- **ACHTUNG: KEIN OVERLOADING**

```
class Car:
    def __init__(self):
        self.brand = None
        self.color = None
        self.horse_power = 0

    def __init__(self, brand, color, horse_power):
        self.brand = brand
        self.color = color
        self.horse_power = horse_power
```



- **Versuchen wir es mal:**

```
car = Car()
```

```
TypeError: __init__() missing 3 required positional arguments: 'brand',  
'color', and 'horse_power'
```

Klassen definieren



- **String-Ausgabe:**

```
class Car:
    def __init__(self, brand, color, horse_power):
        self.brand = brand
        self.color = color
        self.horse_power = horse_power
```

```
my_car = Car("VW", "YELLOW", 75)
print(my_car)
```

- **Versuchen wir es mal:**

```
<__main__.Car object at 0x1036431c0>
```




Wie ist es in Java?



- **String-Ausgabe mit `__str__()` analog zu `toString()`**

```
class Car:
    def __init__(self, brand, color, horse_power):
        self.brand = brand
        self.color = color
        self.horse_power = horse_power

    def __str__(self):
        return f"Marke: {self.brand} / Farbe: {self.color} /" + \
               f" PS: {self.horse_power}"
```

```
my_car = Car("VW", "YELLOW", 75)
print(my_car)
```

- **Versuchen wir die Ausgabe erneut:**

Marke: VW / Farbe: YELLOW / PS: 75



- String-Ausgabe mit `__str__()` und `__repr__()` für die Kommandozeile

```
class Car:
```

```
    def __init__(self, brand, color, horse_power):  
        self.brand = brand  
        self.color = color  
        self.horse_power = horse_power
```

```
    def __str__(self):  
        return f"Marke: {self.brand} / Farbe: {self.color} /" + \  
            f" PS: {self.horse_power}"
```

```
    def __repr__(self):  
        return self.__str__()
```

```
>>> my_birthday = datetime.date(1971, 7, 2)  
>>> my_birthday  
datetime.date(1971, 7, 2)  
>>> str(my_birthday)  
'1971-07-02'
```

- `__str__()` soll menschenlesbar sein
- `__repr__()` soll gut maschinenlesbar und als Objekt rekonstruierbar sein (z. B. Listen)



- **Bekannte Punkt-Notation zum Lesen und Schreiben**

```
print(my_car.brand)
```

```
my_car.color = "STEALTH_BLACK"  
my_car.horse_power += 250  
print(my_car)
```

- **Wir erhalten folgendes:**

```
VW  
Marke: VW / Farbe: STEALTH_BLACK / PS: 325
```

- **ACHTUNG: Es gibt in Python keine wirkliche Sichtbarkeitseinschränkung!!**
-

Verhalten definieren



```
class Car:
    def __init__(self, brand, color, horse_power):
        self.brand = brand
        self.color = color
        self.horse_power = horse_power

    def __str__(self):
        return f"Marke: {self.brand} / Farbe: {self.color} /\n" + \
            f" PS: {self.horse_power}"

    def __repr__(self):
        return self.__str__()

    def paint_with(self, new_color):
        self.color = new_color

    def apply_tuning_kit(self):
        self.horse_power += 150
```

Statische Attribute und Methoden



```
class StaticExample:
    static_info = "Class wide info"

    @staticmethod
    def generate_info():
        print("static methods can be called without creating objects")
        return "Special Information"

    def object_method(self):
        print("object methods must be called on objects")
        print("static methods/variables are accessible")
        return StaticExample.static_info
```

Objekte vergleichen



```
toms_car = Car("Audi", "BLUE", 275)  
jims_car = Car("Audi", "BLUE", 275)  
  
print(toms_car == jims_car)
```



**Was ist das
Ergebnis?**

Objekte vergleichen



- **Gegeben seien folgende Definitionen:**

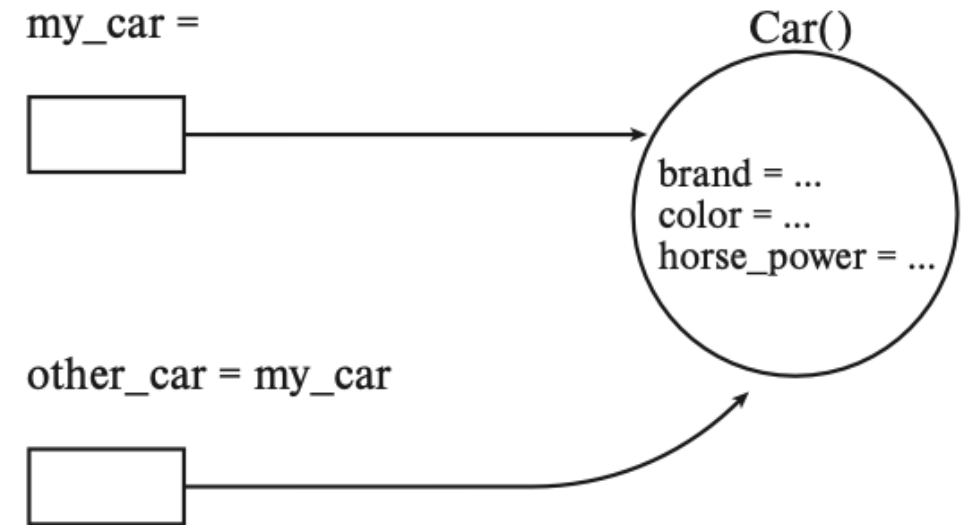
```
toms_car = Car("Audi", "BLUE", 275)
jims_car = Car("Audi", "BLUE", 275)
```

```
print(toms_car == jims_car)
```

- **Referenzgleichheit / Identität**

```
print(id(toms_car))
print(id(jims_car))
```

- **Inhaltliche Gleichheit / semantische Gleichheit => gleiche Werte der Attribute**





**Was braucht es für
semantische Gleichheit
in Java?**

Objekte vergleichen



- Operator 'is' – Mit dem Operator 'is' werden Referenzen verglichen. Somit wird auf *Identität* geprüft, also, ob es sich um dieselben Objekte handelt.
 - Operator '==' – Mit dem Operator '==' werden zwei Objekte bezüglich ihres *Zustands* (d. h. der Wertebelegung der Attribute) verglichen. Zum inhaltlichen Vergleich eigener Klassen ist dort die Methode `__eq__()` selbst zu implementieren.
 - Standardmäßig erfolgt sonst lediglich ein Vergleich der beiden Referenzen mit dem Operator 'is'.
 - In eigenen Realisierungen muss derjenige Teil des Objektzustands verglichen werden, der für die semantische Gleichheit, also die inhaltliche Gleichheit, relevant ist.
-



Art		Java		Python
Referenzgleichheit / Identität		<code>==</code>		<code>is</code>
Semantische / inhaltliche Gleichheit		<code>equals()</code>		<code>==</code>

Implementierung von `__eq__()`



1. **Typprüfung** – Um nicht Äpfel mit Birnen zu vergleichen, sichern wir vor dem inhaltlichen Vergleich ab, dass nur Objekte des gewünschten Typs verglichen werden.
2. **Objektvergleich** – Anschließend werden diejenigen Attributwerte verglichen, die für die Aussage »Gleichheit« relevant sind.
 - Die hierzu notwendigen Attribute des Objekts werden (z. B. in der Reihenfolge ihrer Definition oder des ersten vermuteten Unterschieds) per Operator `==` auf Gleichheit geprüft.

```
def __eq__(self, other):  
    if not isinstance(other, Car):  
        return False  
  
    return self.brand == other.brand and \  
           self.color == other.color and \  
           self.horse_power == other.horse_power
```



DEMO

`car6_comparison.java`
`car7_comparison.py`



**Wie sieht es denn mit
Information Hiding aus?**



Während Sprachen wie Java oder C++ sogenannte Sichtbarkeiten besitzen, um den Zugriff auf private Klassenbestandteile steuern und schützen zu können, ist dies in Python so nicht möglich!



Allerdings gibt es mit `_` und `__` zwei Varianten, um Ähnliches zu erzielen ...

- `_` – Wenn Attribute und Methoden in Python mit `_` beginnen, dann bedeutet der eine Unterstrich per Konvention, dass diese Methode oder dieses Attribut als privat und Implementierungsdetail der Klasse anzusehen ist.
 - Allerdings wird dies von Python nicht forciert und es werden auch keine Zugriffe unterbunden. Vielmehr ist man auf die Beachtung durch andere Programmierer angewiesen.
 - `__` – Ein doppelter Unterstrich kennzeichnet eine spezielle interne Methode, wie wir dies etwa für `__str__()` oder `__eq__()` bereits gesehen haben.
 - Für Attribute eingesetzt, ist dieses Attribut nicht mehr unter seinem Namen nach außen für andere Klassen sichtbar.
 - In beiden Fällen wird der Name dann zu `_ClassName__method/attribute`. Für Methoden dient dies dazu, diese in Vererbungshierarchien speziell zugreifbar zu machen.
-

Information Hiding



```
class Car:
    def __init__(self, brand, color, horse_power):
        self.__brand = brand
        self.__color = color
        self.__horse_power = horse_power

    def paint_with(self, new_color):
        self.__color = new_color

    def apply_tuning_kit(self):
        self.__horse_power += 150

    # Zugriff nach außen gewähren
    def brand(self):
        return self.__brand

    def horse_power(self):
        return self.__horse_power

...
```

Information Hiding – «private Methoden» => Properties



```
class Car:
    def __init__(self, brand, color, horse_power):
        self.__brand = brand
        self.__color = color
        self.__horse_power = horse_power

    def paint_with(self, new_color):
        self.__color = new_color

    def apply_tuning_kit(self):
        self.__horse_power += 150

    def __horse_power(self):
        return self.__horse_power

    def __brand(self):
        return self.__brand
```

...

Information Hiding – «private Methoden» => Properties



...

```
def __get_brand(self):  
    print("__get_brand")  
    return self.__brand
```

```
def __get_color(self):  
    print("__get_color")  
    return self.__color
```

```
def paint_with(self, color):  
    print("paint_with")  
    self.__color = color
```

```
# property(fget, fset, fdel, doc)
```

```
brand = property(__get_brand)
```

```
color = property(__get_color, paint_with)
```

```
horse_power = property(__get_horse_power, __set_horse_power)
```



READ ONLY
READ WRITE
READ WRITE

Information Hiding – «private Methoden» => Properties



```
my_car = Car("RENAULT", "PETROL", 195)
#my_car.brand = "VW" # AttributeError: can't set attribute
my_car.color = "RED"
my_car.horse_power = 7271
```





Die Kapselung ermöglicht es uns, innerhalb von Methoden verschiedene Prüfungen zu integrieren und ungültige Werte zurückzuweisen.

```
def set_horse_power(self, horse_power):  
    if horse_power <= 0 or horse_power > 2_000:  
        raise ValueError("INVALID PS: not in range 1 - 2000")  
  
    self.__horse_power = horse_power
```

Wertebereichsprüfungen mit Properties



Man kann auch Properties mit @ verwenden:

```
@property
def horse_power(self):
    return self.__horse_power

@horse_power.setter
def horse_power(self, horse_power):
    if horse_power <= 0 or horse_power > 2_000:
        raise ValueError("INVALID PS: not in range 1 - 2000")

    self.__horse_power = horse_power
```

Damit sieht es wie ein Attributzugriff aus, bietet aber mehr Möglichkeiten:

```
my_car = Car("Audi", "BLUE", 275)
my_car.horse_power = 1234
print(my_car.horse_power)
```



DEMO

`car_properties1/2.py`
`car8_information_hiding.py`

AttributeError: 'Car' object has no attribute '__brand'
print(toms_car.__brand)



```
class BaseClass:
    def method(self):
        print("called method")

    def other_method(self):
        pass

class SubClass(BaseClass):
    def method(self):
        super().method()
        # weitere Aktionen
        print("other actions")

    # Zusätzliche Funktionalität
    def additional_method(self):
        pass
```


Vererbung

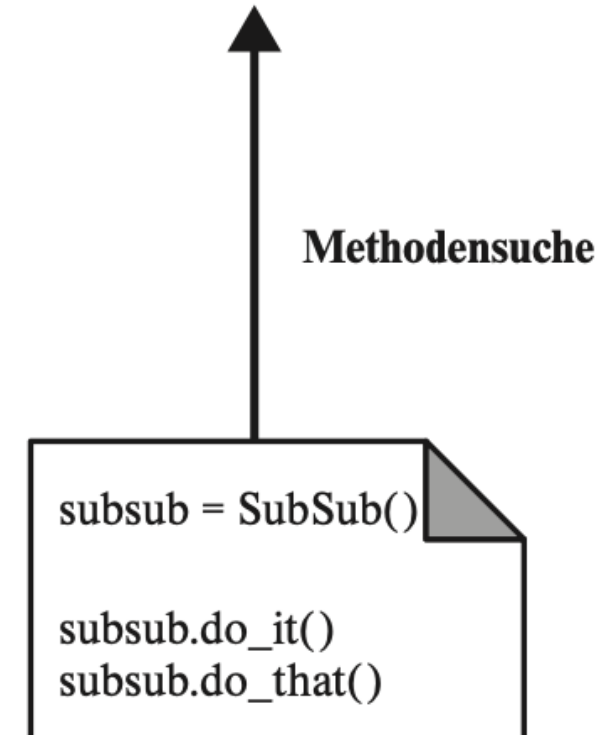
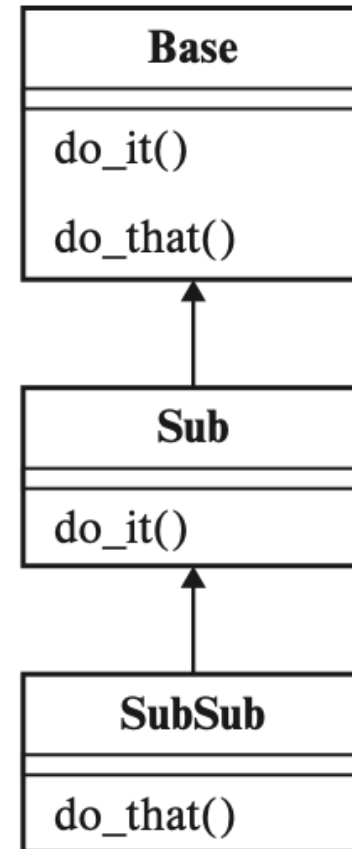


```
class Base:
    def do_it(self):
        pass

    def do_that(self):
        pass

class Sub(Base):
    def do_it(self):
        print("do_it in Sub")

class SubSub(Sub):
    def do_that(self):
        print("do_that in SubSub")
```



Vererbung prüfen – isinstance()



```
class Base:
    def do_it(self):
        pass

    def do_that(self):
        pass

class Sub(Base):
    def do_it(self):
        print("do_it in Sub")

class SubSub(Sub):
    def do_that(self):
        print("do_that in SubSub")
```

```
print(isinstance(Sub, Base))
print(isinstance(SubSub, Base))
print(isinstance(SubSub, Sub))
```

Aufzählungen mit Enum



```
from enum import Enum, auto
```

```
class Jahreszeiten(Enum):  
    FRÜHLING = auto()  
    SOMMER = auto()  
    HERBST = auto()  
    WINTER = auto()
```

```
class Size(Enum):  
    XS = auto()  
    S = auto()  
    M = auto()  
    L = auto()  
    XL = auto()  
    XXL = auto()
```

Aufzählungen mit Enum



```
from enum import Enum, auto
```

```
class Jahreszeiten(Enum):  
    FRÜHLING = auto()  
    SOMMER = auto()  
    HERBST = auto()  
    WINTER = auto()
```

```
class Size(Enum):  
    XS = auto()  
    S = auto()  
    M = auto()  
    L = auto()  
    XL = auto()  
    XXL = auto()
```

```
shirt_size = Size.XL  
print(shirt_size)
```

```
for name in Jahreszeiten:  
    print(name)  
    print(name.value)
```

```
Size.XL  
Jahreszeiten.FRÜHLING  
1  
Jahreszeiten.SOMMER  
2  
Jahreszeiten.HERBST  
3  
Jahreszeiten.WINTER  
4
```

Enum



```
from enum import Enum
```

```
class Direction(Enum):
```

```
    N = (0, -1)
```

```
    NE = (1, -1)
```

```
    E = (1, 0)
```

```
    SE = (1, 1)
```

```
    S = (0, 1)
```

```
    SW = (-1, 1)
```

```
    W = (-1, 0)
```

```
    NW = (-1, -1)
```

```
print(Direction.NE.value)
```

```
ne = Direction.NE
```

```
print(ne.value[0], "/", ne.value[1])
```

```
(1, -1)
```

```
1 / -1
```

Named Tuple



- Datenbehälterklassen

```
class Point2D:  
    def __init__(self, x, y):  
        self.x = x  
        self.y = y
```

```
origin = Point2D(0, 0)  
print(origin)
```

<__main__.Point2D object at 0x102d09fd0>

Name des Typs

Name in Konsolenausgabe

- Named Tuple

```
Point2D = namedtuple('Point2D', ['x', 'y'])  
origin = Point2D(0, 0)  
print(origin)
```

Point2D(x=0, y=0)

Named Tuple – Definitionsvarianten



```
Point2D = namedtuple('Point2D', ['x', 'y'])
```

- **Angaben der Attribute als Tupel**

```
Point2D = namedtuple('Point2D', ('x', 'y'))
```

- **Als String mit kommaseparierter Angabe**

```
Point2D = namedtuple('Point2D', ('x, y'))
```

- **Als reiner String mit Space getrennt:**

```
Point2D = namedtuple('Point2D', ('x y'))
```

Named Tuple / Java Records



- **JAVA**

```
jshell> record Person(String name, int age) {}  
|   created record Person
```

- **PYTHON**

Named Tuple vs Class

```
from collections import namedtuple
```

```
Person = namedtuple('Person', ['name', 'age'])
```

- **Dabei handelt es sich um eine spezielle Unterklasse eines Tupels, die programmgesteuert basierend auf den Angaben zu den benannten Attributen erstellt wird.**

Named Tuple Vorteile am Beispiel



- Bieten eine hilfreiche String-Darstellung
- Unterstützen den indizierten Zugriff
- Unveränderliche Datenstrukturen

```
from collections import namedtuple
Top3 = namedtuple('Top3', ['first', 'second', 'third'])
pizza_top3 = Top3("Funghi", "Diavola", "Napoli")
print(pizza_top3)
print(pizza_top3[0], pizza_top3[1], pizza_top3[2])
pizza_top3[0] = "GYROS"
```

• =>

```
Top3(first='Funghi', second='Diavola', third='Napoli')
Funghi Diavola Napoli
Traceback (most recent call last): ...
    pizza_top3[0] = "GYROS"
TypeError: 'Top3' object does not support item assignment
```



Exercises Part 3

<https://github.com/Michaeli71/Python-Workshop>





PART 4:

Collections

- Schnelleinstieg list
 - Comprehensions
 - Slicing – Zugriff auf Teilbereiche
 - Schnelleinstieg set
 - Schnelleinstieg dict
-



Listen





- Erzeugen und Konkatenieren // Java-Analogie: Collection-Factory-Methoden**

```
>>> list()
[]
>>>
>>> numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>>
>>> zeros = [0] * 10
>>> zeros
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
>>>
>>> zeros += [None] * 5
>>> zeros
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, None, None, None, None, None]
```

- Heterogene Zusammensetzung möglich (wie früher in Java ohne Generics)**

```
>>> [0, "ABC", 42.195, True]
[0, 'ABC', 42.195, True]
```

Listen / Sequentielle Datentypen (wie schon bei Strings)



- **Auf Elemente zugreifen**

```
>>> numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> numbers[0]
0
>>> numbers[7]
7
```

- **Besonderheit: Negative Indizes möglich ($-\text{idx} \Leftrightarrow \text{len}(\text{txt}) - \text{idx}$)**

```
>>> numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> numbers[-1]
9
>>> numbers[-2]
8
>>> numbers[-9]
1
>>> numbers[-10]
0
```



```
>>> numbers[len(content) - 1]
9
>>> numbers[len(content) - 2]
8
>>> numbers[len(content) - 9]
1
>>> numbers[len(content) - 10]
0
```



- **Bereichsverletzung (auch bei negativem Index)**

```
>>> numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>>
```

```
>>> numbers[42]
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
IndexError: list index out of range
```

```
>>>
```

```
>>> numbers[-42]
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
IndexError: list index out of range
```



- **positionsbasiert durchlaufen**

```
>>> values = [0, "ABC", 42.195, True]
>>> for i in range(len(values)):
...     print(i, values[i], end=',')
```

0 0,1 ABC,2 42.195,3 True,

- **elementbasiert**

```
>>> for value in values:
...     print(value, end=',')
```

0,ABC,42.195,True,

- **Kombination: Mit Position und aktuellem Element**

```
for i, value in enumerate(values):
    print(i, value, end=',')
```

0 0,1 ABC,2 42.195,3 True,

- **Kombination mit `sorted()` / `reversed()` wie bei Schleifen möglich**

Beispiel: Konkatination von Listen



Ziel ist eine Ausgabe von 1st of May usw.

Dazu ist eine Liste geeignet zu befüllen:

```
endings = ["st", "nd", "rd"] + 17 * ["th"] + \
           ["st", "nd", "rd"] + 7 * ["th"] + \
           ["st"]
```

```
for i in range(1, 32):
    print(str(i) + endings[i - 1])
```

1st
2nd
3rd
4th
5th
6th
7th
8th
9th
10th
...
21st
22nd
23rd
24th
...
30th
31st



- **Basisfunktionalitäten**

```
numbers = [11, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print("len:", len(numbers))  
print("min:", min(numbers))  
print("max:", max(numbers))  
print("sum:", sum(numbers))
```



```
len: 9  
min: 2  
max: 11  
sum: 55
```

- **Nettigkeiten**

```
numbers.reverse()  
print("reverse:", numbers)  
numbers.sort()  
print("sort:", numbers)
```



```
reverse: [9, 8, 7, 6, 5, 4, 3, 2, 11]  
sort: [2, 3, 4, 5, 6, 7, 8, 9, 11]
```

- **Alternativen: reversed() / sorted()**



- **Hinzufügen**

```
names = []
```

```
names.append("Tim")  
names.append("Tom")  
print(names)
```

```
['Tim', 'Tom']
```

```
names.insert(0, "Anton")  
names.insert(0, "Andreas")  
print(names)
```

```
['Andreas', 'Anton', 'Tim', 'Tom']
```

```
names.insert(4, "Last")  
print(names)
```

```
['Andreas', 'Anton', 'Tim', 'Tom', 'Last']
```



- **Ausflug Fibonacci-Zahlen (rekursive Definition):**

```
>>> def fib(n):  
...     if n <= 1:  
...         return n  
...     return fib(n-2) + fib(n-1)  
...  
>>> fib(5)  
5
```

- **Fibonacci Berechnungs-Trick mit negativem Index:**

```
>>> fib_numbers = [0, 1]  
>>> for i in range(10):  
...     fib_numbers.append(fib_numbers[-2] + fib_numbers[-1])  
...  
>>> fib_numbers  
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```



- **Ändern**

```
names[1] = "Mike"  
print(names)
```

```
['Andreas', 'Mike', 'Tim', 'Tom', 'Last']
```

- **Löschen**

```
del names[2]  
print(names)
```

```
['Andreas', 'Mike', 'Tom', 'Last']
```

```
names.remove("Tom")  
print(names)
```

```
['Andreas', 'Mike', 'Last']
```

```
names.clear()  
print(names)
```

```
[]
```

```
names.remove("Tim")
```

ValueError: list.remove(x): x not in list



- **Multi-Append (+= / extend())**

```
cities = ["Zürich", "Luzern"]  
cities += ["Bremen", "Aachen"]  
print(cities)
```

```
cities.extend(["Paris", "London"])  
print(cities)
```

Fallstrick bei sequentiellen Typen

```
cities.extend("OSLO")  
cities += "KIEL"  
print(cities)
```

- =>

```
['Zürich', 'Luzern', 'Bremen', 'Aachen']  
['Zürich', 'Luzern', 'Bremen', 'Aachen', 'Paris', 'London']  
['Zürich', 'Luzern', 'Bremen', 'Aachen', 'Paris', 'London', 'O', 'S', 'L', 'O',  
'K', 'I', 'E', 'L']
```



- **Enthaltensein prüfen (in)**

```
>>> values = (0, "ABC", 42.195, True)
>>>
>>> "ABC" in values
True
>>>
>>> ["ABC", 42.195] in values
False
```



Slicing





- **Slicing => Extraktion einer neuen Liste von Position startidx bis endidx**

`text[startidx:endidx]`

- **Beispiel**

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
print(numbers[2:8])  
print(numbers[0:3])  
print(numbers[-3:])
```



```
[2, 3, 4, 5, 6, 7]  
[0, 1, 2]  
[10, 11, 12]
```



- **Slicing – Besonderheiten**

`text[startidx:endidx:step]`

- **Beispiel**

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
print(numbers[1::2])  
print(numbers[2:10:2])  
print(numbers[10:2:-2])
```



```
[1, 3, 5, 7, 9, 11]  
[2, 4, 6, 8]  
[10, 8, 6, 4]
```



- Slicing – Trickserei

text[startidx:endidx:step] = <WERTE>

```
numbers = [0, 1, 2, 3, 9, 10, 11, 12]
print(numbers)
numbers[4:4] = [4, 5, 6, 7, 8]
print(numbers)
numbers[4:8] = [44, 55, 66, 77, 88, "-", "-", "-"]
print(numbers)
numbers[0:14:2] = ["X1", "X2", "X3", "X4", "X5", "X6", "X7"]
print(numbers)
```



```
[0, 1, 2, 3, 9, 10, 11, 12]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
[0, 1, 2, 3, 44, 55, 66, 77, 88, '-', '-', '-', 8, 9, 10, 11, 12]
['X1', 1, 'X2', 3, 'X3', 55, 'X4', 77, 'X5', 8, 'X6', 10, 'X7', 12]
```



List Comprehensions



List Comprehension



- **Berechnungsvorschrift angeben**

[<expr> for value in <iterables>]

- **Beispiel**

```
>>> [value for value in range(10)]  
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
>>>  
>>> [value * value for value in range(10)]  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

List Comprehension



- **Berechnungsvorschrift inklusive Bedingung angeben**

[<expr> for value in <iterables> if <condition>]

- **Beispiel**

```
>>> [value * 2 for value in range(10) if value % 2 != 0]
[2, 6, 10, 14, 18]
>>>
>>> [value ** 2 for value in range(10) if value % 2 == 0]
[0, 4, 16, 36, 64]
```

List Comprehension



- **«Komplexere» Berechnungsvorschrift und mehrere Schleifen**

```
>>> [[x, y * 2] for x in range(3) for y in range(5)]
```

- **Ohne Comprehension**

```
>>> result = []
>>> for x in range(3):
...     for y in range(5):
...         result.append([x, y * 2])
```

- **=>**

```
[[0, 0], [0, 2], [0, 4], [0, 6], [0, 8],
 [1, 0], [1, 2], [1, 4], [1, 6], [1, 8],
 [2, 0], [2, 2], [2, 4], [2, 6], [2, 8]]
```



DEMO

`list_*.py`



Tupel



Tupel



- **Nahezu wie Listen, aber unveränderlich**

```
>>> (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
>>>
>>> (0, "ABC", 42.195, True)
(0, 'ABC', 42.195, True)
>>> (1)
1
>>> (1,)
(1,)
```

- **Aber Achtung: KEINE Tuple Comprehension!**

```
>>> (value for value in range(10))
<generator object <genexpr> at 0x1050b7220>
```

Tuple Packing / Unpacking



- **Mehrere Werte (als Tupel) mehreren Variablen zuweisen (Unpacking)**

```
>>> x, y, z = (1, 2, 3)
>>> print(x, y, z)
1 2 3
```

- **Mehrere Werte einer Variablen als Tupel zuweisen (Packing)**

```
>>> values = 1, 2, 3
>>> values
(1, 2, 3)
```

- **SWAP-Trick**

```
>>> x, y = y, x
>>> print(x, y, z)
2 1 3
```

Tuple Unpacking bei Dicts



- **Mehrere Werte (als Tupel) mehreren Variablen zuweisen (Unpacking)**

```
cities_inhabitants = {"Kiel": 250_000,  
                     "Bremen": 550_000,  
                     "Zürich": 400_000}
```

```
city, inhabitant_count = cities_inhabitants.popitem()  
print(city, inhabitant_count)
```

Unpacking

```
for key, value in cities_inhabitants.items():  
    print(key, value)
```

- =>

```
Zürich 400000  
Kiel 250000  
Bremen 550000
```

Tuple Unpacking



- **Problem beim Unpacking, wenn zu viele Werte**

```
>>> first, second = (1, 2, 3, 4, "END")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 2)
```

- **Trick, falls zu viele Werte**

```
>>> first, second, *rest = (1, 2, 3, 4, "END")
>>> print(first, second, rest)
1 2 [3, 4, 'END']
```

Tuple Unpacking



- **Trick, falls zu viele Werte**

```
>>> first, second, *rest = (1, 2, 3, 4, "END")
>>> print(first, second, rest)
1 2 [3, 4, 'END']
```

- **Alternative: Mitte oder gar vorne auffüllen**

```
>>> first, *rest, third = (1, 2, 3, 4, "END")
>>> print(first, rest, third)
1 [2, 3, 4] END
>>>
>>> *rest, first, second = (1, 2, 3, 4, "END")
>>> print(rest, first, second)
[1, 2, 3] 4 END
```



Mengen (set)



Sets – Basisfunktionalitäten



- Ein set ist eine Sammlung (Menge) von Elementen.
- mathematische Konzept => keine Duplikate
- Somit bilden Sets eine ungeordnete, duplikatfreie Datenstruktur, aber bieten keinen indizierten Zugriff.
- Stattdessen einige Mengenoperationen insbesondere die Berechnung von Vereinigungs-, Schnitt-, Differenz- und symmetrischen Differenzmengen:





- **Erzeugen // Java-Analogie: Collection-Factory-Methoden**

```
>>> set()
set()
>>>
>>> numbers = {0, 1, 2, 3, 4, 5, 6, 7}
>>> numbers
{0, 1, 2, 3, 4, 5, 6, 7}
>>> type(numbers)
<class 'set'>
```

- **Heterogene Zusammensetzung möglich (wie früher in Java ohne Generics)**

```
>>> {0, "ABC", 42.195, True}
{0, True, 42.195, 'ABC'}
```



- **Basisfunktionalitäten**

```
numbers = {11, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
print("len:", len(numbers))  
print("min:", min(numbers))  
print("max:", max(numbers))  
print("sum:", sum(numbers))
```



```
len: 9  
min: 2  
max: 11  
sum: 55
```

- **elementbasiert durchlaufen**

```
>>> values = {0, "ABC", 42.195, True}
```

```
>>> for value in values:  
...     print(value, end=',')
```



```
0,ABC,42.195,True,
```



- **Hinzufügen**

```
names = set()
names.add("Tim")
names.add("Tom")
print(names)
```

{'Tim', 'Tom'}

```
names.add("Tim")
names.add("Last")
print(names)
```

{'Tim', 'Last', 'Tom'}

```
names.update(["Mike", "Peter", "John"])
print(names)
```

{'Mike', 'Tom', 'John', 'Tim', 'Peter', 'Last'}

- ***Fallstrick bei sequentiellen Typen***

```
names.update("Michael")
print(names)
```

{'John', 'i', 'Mike', 'l', 'M', 'Peter', 'e', 'Tom', 'a', 'c', 'Last', 'Tim', 'h'}



- **Löschen**

```
for letter in "Michael":  
    names.remove(letter)  
print(names)
```

```
names.remove("Tom")  
print(names)
```

```
names.clear()  
print(names)
```

```
names.remove("Tim")
```

```
{'Last', 'Mike', 'John', 'Tim', 'Peter', 'Tom'}
```

```
{'Peter', 'John', 'Tim', 'Mike'}
```

```
set()
```

```
KeyError: 'Tim'
```



- **Keine Methoden `reverse()` / `sort()` zur Sortierung und zum Umdrehen der Reihenfolge, auch kein `reversed()`, aber `sorted()` wird unterstützt**

```
>>> names = {'Peter', 'John', 'Tim', 'Mike'}  
>>> for elem in sorted(names):  
...     print(elem)  
...  
John  
Mike  
Peter  
Tim
```

- **Enthaltensein prüfen (`in`)**

```
>>> values = {0, "ABC", 42.195, True}  
>>>  
>>> "ABC" in values  
True  
>>>  
>>> {"ABC", 42.195} in values  
False
```



**Was macht man also
mit Prüfungen von
Wertefolgen?**

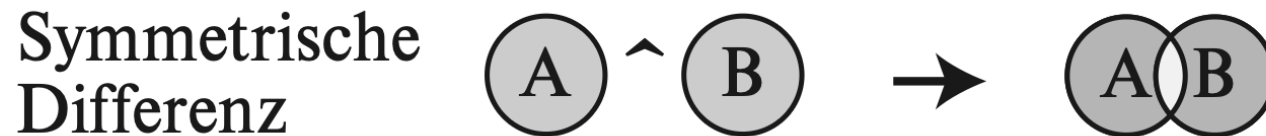
=>

Mengenoperationen!





- **Mengenoperationen: Berechnung von Vereinigungs-, Schnitt-, Differenz- und symmetrischen Differenzmengen:**



Sets – Mengenoperationen

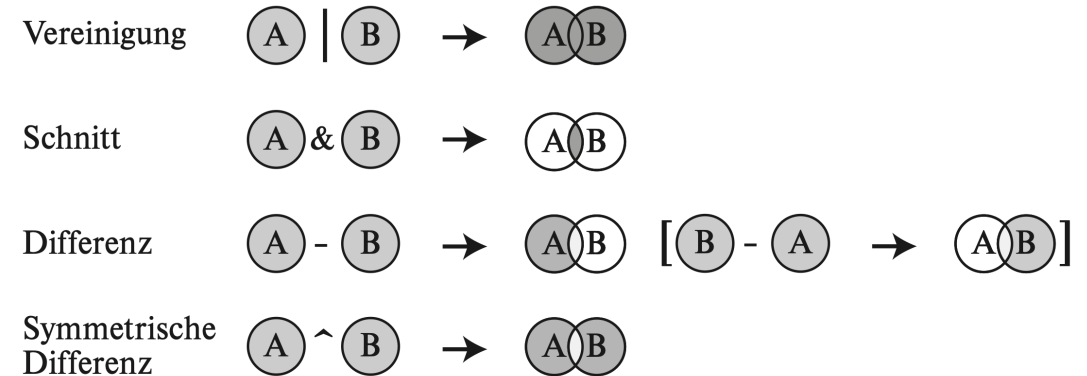


- **Mengenoperationen**

```
number_set1 = {1, 2, 3, 4, 5, 6, 7, 8}
number_set2 = {2, 3, 5, 7, 9, 11, 13}
print("union: %s\nintersection: %s" \
      "\ndiff 1-2: %s\ndiff 2-1: %s" \
      "\nsym diff: %s" %
      ((number_set1 | number_set2),
       (number_set1 & number_set2),
       (number_set1 - number_set2),
       (number_set2 - number_set1),
       (number_set1 ^ number_set2)))
```

- =>

```
union: {1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 13}
intersection: {2, 3, 5, 7}
diff 1-2: {8, 1, 4, 6}
diff 2-1: {9, 11, 13}
sym diff: {1, 4, 6, 8, 9, 11, 13}
```





- **Kommen wir zur Subset-Prüfung zurück**
- **Wenn $A \& B = B \Rightarrow$ dann ist es ein Subset, enthält also alle Elemente von B**

```
>>> {"ABC", 42.195} & {0, "ABC", 42.195, True} == {"ABC", 42.195}
True
```

- **Praktischerweise existieren zur Abfrage bereits passende vordefinierte Methoden:**

```
>>> print({"ABC", 42.195}.issubset({0, "ABC", 42.195, True}))
True
>>>
>>> print({0, "ABC", 42.195, True}.issuperset({"ABC", 42.195}))
True
```



Dictionaries (dict)





- Erzeugen (leer und mit vordefinierten Werten) // Java-Analogie: Collection-Factory-Methoden

```
>>> dict()
{}
>>>
>>> {}
{}
>>> type({})
<class 'dict'>
>>>
>>> city_inhabitants_map = {}
>>>
>>> bern_berlin_map = {"Bern": 170_000, "Berlin": 3_500_000}
>>> bern_berlin_map
{'Bern': 170000, 'Berlin': 3500000}
```



- **Hinzufügen**

```
city_inhabitants_map = {}
```

```
city_inhabitants_map["Zürich"] = 400_000
```

```
city_inhabitants_map["Hamburg"] = 2_000_000
```

```
city_inhabitants_map["Kiel"] = 250_000
```

```
print(city_inhabitants_map)
```

```
{'Zürich': 400000, 'Hamburg': 2000000, 'Kiel': 250000}
```

```
city_inhabitants_map.update({"Bern": 170_000, "Berlin": 3_500_000})
```

```
print(city_inhabitants_map)
```

```
{'Zürich': 400000, 'Hamburg': 2000000, 'Kiel': 250000,  
'Bern': 170000, 'Berlin': 3500000}
```

Dicts – Durch die Elemente iterieren



- Bei Dictionaries existiert kein indizierter Zugriff
- Allerdings lassen sich die Abbildungen mit folgenden Methoden auslesen:
 1. `items()` – Erzeugt eine Liste, die alle Schlüssel-Wert-Paare des Dictionaries als Tupel.
 2. `keys()` – Liefert eine Liste mit allen im Dictionary hinterlegten Schlüsseln.
 3. `values()` – Liefert eine Liste mit allen im Dictionary hinterlegten Werten.

```
for key, value in city_inhabitants_map.items():  
    print("The city of", key, "has", value, "inhabitants")
```

Dicts – Durch die Elemente iterieren



- **Beispiel:**

```
for key, value in city_inhabitants_map.items():  
    print("The city of", key, "has", value, "inhabitants")  
  
print("-".join(city_inhabitants_map.keys()))  
print("Gesamteinwohner:", sum(city_inhabitants_map.values()))
```

- =>

```
The city of Zürich has 400000 inhabitants  
The city of Hamburg has 2000000 inhabitants  
The city of Kiel has 250000 inhabitants  
The city of Bern has 170000 inhabitants  
The city of Berlin has 3500000 inhabitants  
Zürich-Hamburg-Kiel-Bern-Berlin  
Gesamteinwohner: 6320000
```



- **Prüfen, ob ein Eintrag existiert (in)**

<pre>print("Zürich" in city_inhabitants_map)</pre>	True
<pre>print("Bremen" in city_inhabitants_map)</pre>	False

- **Prüfen, ob ein Wert existiert (in <values>)**

<pre>print(400_000 in city_inhabitants_map.values())</pre>	True
<pre>print(1_234_567 in city_inhabitants_map.values())</pre>	False



- **Modifikation ausführen**

```
city_inhabitants_map["Bern"] = 144_000
```

- **Löschen**

```
del city_inhabitants_map["Bern"]  
print(city_inhabitants_map)
```

```
{'Zürich': 400000, 'Hamburg': 2000000,  
'Kiel': 250000, 'Berlin': 3500000}
```

```
city_inhabitants_map.pop("Kiel")  
print(city_inhabitants_map)
```

```
{'Zürich': 400000, 'Hamburg': 2000000,  
'Berlin': 3500000}
```

```
city_inhabitants_map.clear()  
print(len(city_inhabitants_map))
```

```
0
```




Mehrdimensionale Listen



Mehrdimensionale Listen



- Abschließend beschäftigen wir uns kurz mit mehrdimensionalen Listen.
- Weil es in der Praxis häufiger vorkommt sowie weil man es sich auch visuell gut vorstellen kann, betrachten wir nur zweidimensionale Listen.

	0	1	2	3	4
0	0,0	0,1	0,2	0,3	0,4
1	1,0	1,1	1,2	1,3	1,4
2	2,0	2,1	2,2	2,3	2,4

- Eine zweidimensionale rechteckige Liste kann man etwa zur Modellierung eines Spielfelds, eines Sudoku-Rätsels oder einer Landschaft nutzen.

```
#####  
##  P  ##  
####  $ X  ####  
##### $ #####  
#####
```

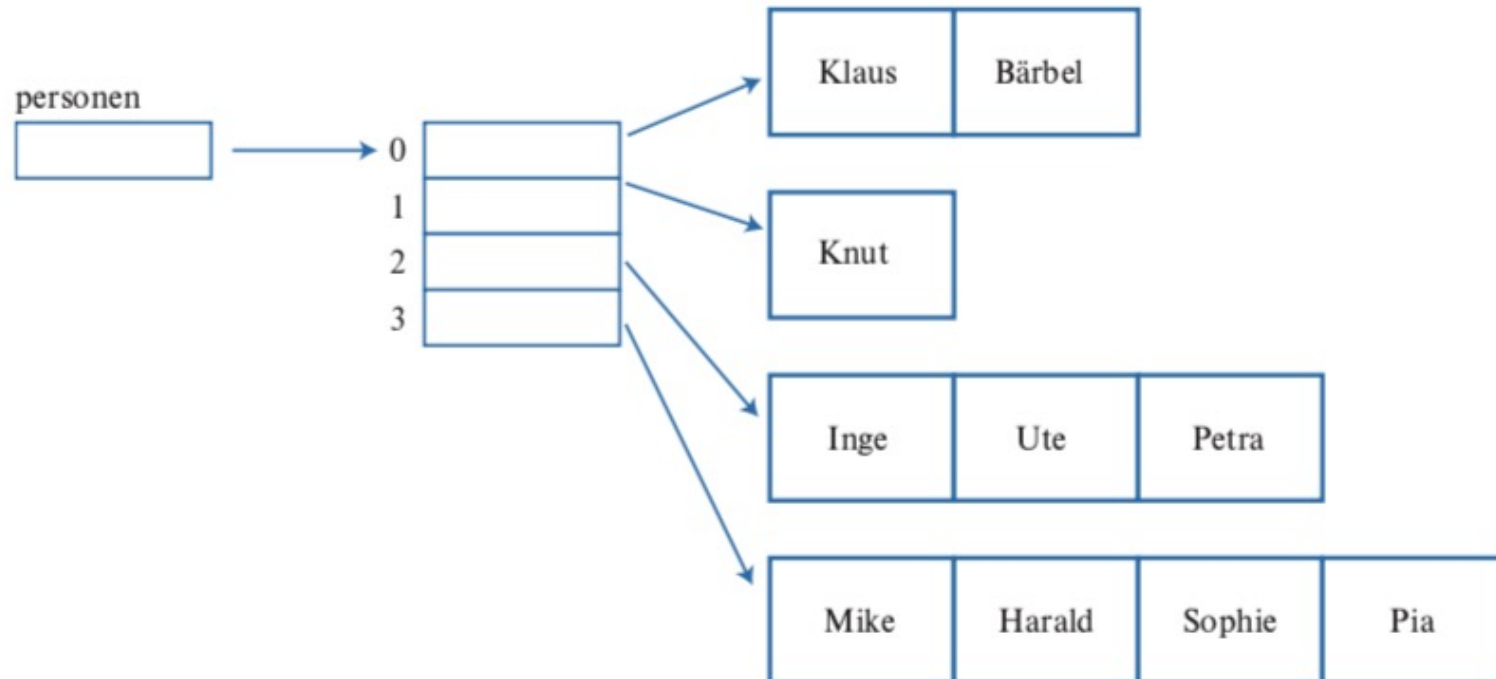
Mehrdimensionale Listen



- **Mehrdimensionale Listen müssen nicht rechteckig sein. Dreieck:**

```
twodim_triangle = [[1],  
                   [2, 2],  
                   [3, 3, 3]]
```

- **Beliebige Verschachtelung:**





Exercises Part 4

<https://github.com/Michaeli71/Python-Workshop>





PART 5:

Collections Advanced

- Sortierung – Analogie Comparator



Sortierung





- **JAVA**

```
jshell> var modifiableNames = new ArrayList<>(names)
modifiable ==> [Mike, Andy, Peter, Jim, Tim]
```

```
jshell> modifiableNames.sort(Comparator.naturalOrder())
```

```
jshell> modifiableNames
modifiableNames ==> [Andy, Jim, Mike, Peter, Tim]
```

- **PYTHON**

```
>>> names = ["Mike", "Andy", "Peter", "Jim", "Tim"]
>>> names.sort()
>>> names
['Andy', 'Jim', 'Mike', 'Peter', 'Tim']
```

Umgekehrte Ordnung



- **JAVA: Reverse order**

```
jshell> modifiableNames.sort(Comparator.reverseOrder())
```

```
jshell> modifiableNames  
modifiableNames ==> [Tim, Peter, Mike, Jim, Andy]
```

- **PYTHON: Reverse order**

```
>>> names.sort(reverse=True)  
>>> names  
['Tim', 'Peter', 'Mike', 'Jim', 'Andy']
```


Spezielle Sortierung nach 3. Buchstaben



- **JAVA: by3ndChar**

```
jshell> modifiableNames.sort(Comparator.comparing(name -> name.charAt(2)))
```

```
jshell> modifiableNames
```

```
modifiableNames ==> [Andy, Mike, Tim, Jim, Peter]
```

- **PYTHON: by3ndChar**

```
>>> names.sort(key=lambda name: name[2])
```

```
>>> names
```

```
['Andy', 'Mike', 'Jim', 'Tim', 'Peter']
```

Spezielle Sortierung nach letztem Buchstaben



- **JAVA: byLastChar**

```
jshell> modifiableNames.sort(  
    Comparator.comparing(name -> name.charAt(name.length() - 1)))
```

```
jshell> modifiableNames  
modifiableNames ==> [Mike, Tim, Jim, Peter, Andy]
```

- **PYTHON: byLastChar**

```
>>> names.sort(key=lambda name: name[-1])  
>>> print(names)  
['Mike', 'Tim', 'Jim', 'Peter', 'Andy']
```

Spezielle Sortierung nach Länge



- **JAVA: Nach Länge**

```
jshell> modifiableNames.sort(Comparator.comparing(String::length))
```

```
jshell> modifiableNames  
modifiableNames ==> [Jim, Tim, Mike, Andy, Peter]
```

- **PYTHON: Nach Länge sowie auch absteigend**

```
>>> names = ["Mike", "Andy", "Peter", "Jim", "Tim"]  
>>>  
>>> names.sort(key=len)  
>>> names  
['Jim', 'Tim', 'Mike', 'Andy', 'Peter']  
>>>  
>>> names.sort(key=len, reverse=True)  
>>> names  
['Peter', 'Mike', 'Andy', 'Jim', 'Tim']
```

Case-Insensitive Sortierung



- **JAVA:**

```
jshell> var names = new ArrayList<>(
    List.of("tim", "TOM", "MIKE", "Michael", "andreas", "STEFAN"))
names ==> [tim, TOM, MIKE, Michael, andreas, STEFAN]
```

```
jshell> names.sort(Comparator.comparing(String::toLowerCase))
```

```
jshell> names
names ==> [andreas, Michael, MIKE, STEFAN, tim, TOM]
```

- **PYTHON:**

```
names = ["tim", "TOM", "MIKE", "Michael", "andreas", "STEFAN"]
```

```
names.sort(key=str.lower)
```

```
print(names)
```

```
names.sort(key=str.casefold)
```

```
print(names)                                ['andreas', 'Michael', 'MIKE', 'STEFAN', 'tim', 'TOM']
```



- Nach Länge, dann alphabetisch durch Angabe eines Tupels

```
values = ["a", "cc", "bbb", "dddd", "ee", "f", "d", "eee"]  
print(sorted(values, key=lambda x: (len(x), x)))
```

- =>

```
['a', 'd', 'f', 'cc', 'ee', 'bbb', 'eee', 'dddd']
```

Mehrfach Sortierung



- **Absteigend nach Länge und aufsteigend / absteigend nach Wert**

```
values = ["a", "cc", "bbb", "dddd", "ee", "f", "d", "eee"]  
print(sorted(values, key=lambda x: (-len(x), x)))
```

```
values = ["a", "cc", "bbb", "dddd", "ee", "f", "d", "eee"]  
print(sorted(values, key=lambda x: (len(x), x), reverse=True))
```

- =>

```
['dddd', 'bbb', 'eee', 'cc', 'ee', 'a', 'd', 'f']  
['dddd', 'eee', 'bbb', 'ee', 'cc', 'f', 'd', 'a']
```

Mehrfach Sortierung



- Nach Anzahl Programmiersprachen absteigend, nach Namen aufsteigend

```
programmers = [("Tim", ["Java"]), ("Tom", ["Java", "Python"]),  
               ("Michael", ["Java", "Python"]), ("Jim", ["Python"]),  
               ("Polyglotti", ["Java", "Python", "C#", "C++"])]
```

```
print(sorted(programmers, key=lambda x: (-len(x[1]), x[0])))
```

- =>

```
[('Polyglotti', ['Java', 'Python', 'C#', 'C++']),  
 ('Michael', ['Java', 'Python']), ('Tom', ['Java', 'Python']),  
 ('Jim', ['Python']), ('Tim', ['Java'])]
```



DEMO

`sorting_*.py`

Datenmodell



```
from collections import namedtuple

Person = namedtuple('Person', ['name', 'age'])

persons = [Person("Mike", 30), Person("Tim", 50),
            Person("Michael", 50), Person("Tom", 30),
            Person("Arne", 7), Person("Jim", 30)]
```

Sortierung – nach einem Kriterium



Alter absteigend

```
persons.sort(key=lambda p: p.age, reverse=True)
print(persons)
```

```
persons.sort(key=lambda p: -p.age)
print(persons)
```

Name absteigend

```
persons.sort(key=lambda p: p.name, reverse=True)
print(persons)
```

#persons.sort(key=lambda p: -p.name)

=>

```
[Person(name='Tim', age=50), Person(name='Michael', age=50), Person(name='Mike', age=30),
Person(name='Tom', age=30), Person(name='Jim', age=30), Person(name='Arne', age=7)]
[Person(name='Tim', age=50), Person(name='Michael', age=50), Person(name='Mike', age=30),
Person(name='Tom', age=30), Person(name='Jim', age=30), Person(name='Arne', age=7)]
[Person(name='Tom', age=30), Person(name='Tim', age=50), Person(name='Mike', age=30),
Person(name='Michael', age=50), Person(name='Jim', age=30), Person(name='Arne', age=7)]
```

Sortierung – nach zwei Kriterien



Name, Alter absteigend

```
persons.sort(key=lambda p: (p.name, -p.age))  
print(persons)
```

Alter, Name

```
persons.sort(key=lambda p: (p.age, p.name))  
print(persons)
```

Alter, Name absteigend

```
persons.sort(key=lambda p: (-p.age, p.name), reverse=True)  
print(persons)
```

=>

```
[Person(name='Arne', age=7), Person(name='Jim', age=30), Person(name='Michael', age=50),  
Person(name='Mike', age=30), Person(name='Tim', age=50), Person(name='Tom', age=30)]
```

```
[Person(name='Arne', age=7), Person(name='Jim', age=30), Person(name='Mike', age=30),  
Person(name='Tom', age=30), Person(name='Michael', age=50), Person(name='Tim', age=50)]
```

```
[Person(name='Arne', age=7), Person(name='Tom', age=30), Person(name='Mike', age=30),  
Person(name='Jim', age=30), Person(name='Tim', age=50), Person(name='Michael', age=50)]
```

Sortierung von Klassen



```
class SimplePerson:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"{self.name} {self.age}"

persons = [SimplePerson("Mike", 50), SimplePerson("Jim", 7),
            SimplePerson("Tim", 50), SimplePerson("John", 14),
            SimplePerson("Peter", 50), SimplePerson("Fred", 14),
            SimplePerson("Peter", 40), SimplePerson("Peter", 55)]

persons.sort()
print("persons", persons)
```

TypeError: '<' not supported between instances of 'SimplePerson' and 'SimplePerson'

Sortierung von Klassen – Natürliche Ordnung



```
class SimplePerson:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"{self.name} {self.age}"

    def __lt__(self, other):
        return self.name < other.name or \
            (self.name == other.name and self.age < other.age)
```

```
persons = [SimplePerson("Mike", 50), SimplePerson("Jim", 7),
            SimplePerson("Tim", 50), SimplePerson("John", 14),
            SimplePerson("Peter", 50), SimplePerson("Fred", 14),
            SimplePerson("Peter", 40), SimplePerson("Peter", 55)]
```

```
persons.sort()
print("persons", persons)
```

persons [Fred 14, Jim 7, John 14, Mike 50, Peter 40, Peter 50, Peter 55, Tim 50]

Sortierung von Klassen – nach einem Kriterium



```
def by_name_and_age(person):  
    return person.name, person.age
```

```
def by_age_and_name(person):  
    return person.age, person.name
```

```
sorted_by_name_and_age = sorted(persons, key=by_name_and_age)  
print(sorted_by_name_and_age)
```

```
sorted_by_age_and_name = sorted(persons, key=by_age_and_name)  
print(sorted_by_age_and_name)
```

=>

```
[Fred 14, Jim 7, John 14, Mike 50, Peter 40, Peter 50, Peter 55, Tim 50]  
[Jim 7, Fred 14, John 14, Peter 40, Mike 50, Peter 50, Tim 50, Peter 55]
```



Exercises Part 5

<https://github.com/Michaeli71/Python-Workshop>





PART 6:

Exception Handling



- **Fehler können beim Programmieren eigentlich immer und überall auftreten:**

```
>>> 7 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

```
>>> names = ["Tim", "Tom", "Mike"]
>>> names[42]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

- **Wenn als Folge des Fehlers eine Exception (oftmals mit Namensendung Error oder Exception) auftritt, dann stoppt die Programmausführung.**
 - **Wichtig ist eine geeignete Reaktion darauf**
-



- **Exceptions sehr ähnlich zu Java mit try-catch in Python mit try-except**
 - **Für verschiedene Arten von Problemen verschiedene Typen von Exceptions. Vier recht gebräuchliche vordefinierte Typen sind diese:**
 - **ValueError** – Mit einem ValueError können falsche Belegungen von Parametern ausgedrückt werden.
 - **IndexError** – Zu einem IndexError kommt es bei fehlerhaften Indexangaben beim Zugriff etwa auf eine Liste.
 - **KeyError** – Mit einem KeyError werden fehlerhafte Zugriffe (nicht existenter Schlüssel) auf Dictionaries ausgedrückt.
 - **ZeroDivisionError** – Auf eine unerlaubte Division durch Null kann mittels einem ZeroDivisionError hingewiesen werden.
-



- Exceptions sehr ähnlich zu Java mit try-catch in Python mit try-except

try:

Hier können Exceptions auftreten

except Exception-Typ1:

Hier können Fehlersituation behandelt werden

except Exception-Typ2:

Hier können Fehlersituation behandelt werden

- **Beispiel**

try:

value = int("7271")

print("after parsing 1. result", value)

value = int("ERROR")

print("after parsing 2. result", value)

except ValueError:

print("can't parse to integer")

after parsing 1. result 7271
can't parse to integer



- **Auf mehrere Exceptions reagieren (Java: Multi Catch)**

```
names = ["Tim", "Tom", "Mike"]
for i in range(5):
    try:
        value = int(names[i])
    except (ValueError, ZeroDivisionError):
        print("can't parse to integer")
    except IndexError:
        print("wrong index")
```

- **Ergebnis**

```
can't parse to integer
can't parse to integer
can't parse to integer
wrong index
wrong index
```



- Auf mehrere Exceptions reagieren und Zugriff auf Exception

```
names = ["Tim", "Tom", "Mike"]
for i in range(5):
    try:
        value = int(names[i])
    except (ValueError, ZeroDivisionError) as ex:
        print("can't parse to integer:", ex)
    except IndexError as ie:
        print("wrong index:", ie)
```

- Ergebnis

```
can't parse to integer: invalid literal for int() with base 10: 'Tim'
can't parse to integer: invalid literal for int() with base 10: 'Tom'
can't parse to integer: invalid literal for int() with base 10: 'Mike'
wrong index: list index out of range
wrong index: list index out of range
```



- Unspezifisch auf mehrere Exceptions reagieren

```
names = ["Tim", "Tom", "Mike"]
try:
    print("INVALID INDEX: " + names[42])
except:
    print("an unspecified error occurred. seems to be wrong index")
```

- Ergebnis

an unspecified error occurred. seems to be wrong index

- **ABER: Mit der gezeigten Art lassen sich Fehlersituationen nicht unterscheiden und somit kann man nicht adäquat auf unterschiedliche Probleme reagieren.**



- **Folgende grundsätzliche Struktur**

```
try:  
    # Hier können Exceptions auftreten  
except:  
    # Hier werden Exceptions abgearbeitet  
finally:  
    # Wird immer durchlaufen, ist allerdings optional
```

- **Beispiel**

```
names = ["Tim", "Tom", "Mike"]  
try:  
    print("INVALID INDEX: " + names[42])  
except:  
    print("wrong index")  
finally:  
    print("ALWAYS EXECUTED")
```



- **Freigabe von Systemressourcen**

```
file = open("test.txt", encoding='utf-8')
try:
    # Datei lesen und verarbeiten
finally:
    file.close()
```

- **Kurzform analog zu Java-ARM mit ContextManager (with)**

```
with open("test.txt", encoding='utf-8') as file:
    # Datei lesen und verarbeiten
```


Exceptions selbst auslösen – raise



- Mittlerweile haben wir schon mehrmals gesehen, dass bei der Abarbeitung von Programmen in gewissen Fehlersituationen automatisch Exceptions ausgelöst werden.
- Aber auch wir als Programmierer können selbst Exceptions auslösen.
- Dazu dient das Schlüsselwort **raise** in Kombination mit einem **Ausnahmetyp**.

```
def ensure_value_in_range(value, lower_bound, upper_bound):  
    if value < lower_bound or value > upper_bound:  
        raise ValueError("out of bounds")  
    pass
```

Eigene Exception-Typen definieren



- Neben der Verwendung vordefinierter Exceptions problemlos möglich, eigene Typen von Exceptions zu definieren.

```
class CustomerNotFoundException(Exception):  
    pass
```

- Sinnvoller als eine derart pure Exception ist es natürlich, dort weitere Informationen bereitzuhalten

```
class CustomerNotFoundException(Exception):  
    def __init__(self, customer_id, customer_name):  
        self.id = customer_id  
        self.name = customer_name  
  
    def __str__(self):  
        return "customer {} with id {} not found".format(self.name, self.id)
```

Eigene Exception-Typen definieren: Exception für Bereichsprüfung



- Wir können natürlich auch die Bereichsprüfung ein wenig netter gestalten, indem wir eine eigene Klasse `ValueOutOfBoundsError` basierend auf `ValueError` definieren:

```
class ValueOutOfBoundsError(ValueError):  
    def __init__(self, value, lower, upper):  
        self.value = value  
        self.lower = lower  
        self.upper = upper  
  
    def __str__(self):  
        return "{} not in {} - {}".format(self.value, self.lower, self.upper)
```

Propagation von Exceptions



- In `func3()` wird ein `ValueError` ausgelöst. In keiner der Funktionen findet eine Fehlerbehandlung statt:

```
def func1():  
    func2()  
  
def func2():  
    func3()  
  
def func3():  
    raise ValueError  
  
def main():  
    func1()
```

- Aufruf: `main()` → `func1()` → `func2()` → `func3()` und Rückpropagation



- **Integration Fehlerbehandlung und Auslösen unterschiedlicher Exceptions
=> Propagation zum passenden except-Block bzw. bis ganz zur main():**

```
def func1(value):  
    func2(value)
```

```
def func2(value):  
    try:  
        func3(value)  
    except ValueError as voobe:  
        print(voobe)
```

```
def func3(value):  
    if not value:  
        raise ValueError  
    raise ValueError(value, 1, 100)
```

```
def main():  
    func1(123)  
    func1(None)
```

```
123 not in 1 - 100  
Traceback (most recent call last):  
...  
    raise ValueError  
ValueError
```



Was ist eigentlich mit Assertions?



- **assert** gibt es in vielen Programmiersprachen
- hilft dabei, Probleme frühzeitig in Programmen erkennen zu können.
- Dazu kann man Absicherungen mit folgender Syntax angeben:
`assert condition`
- Äquivalent zu
`if not condition:
 raise AssertionError()`



- **Einsatzgebiete**

- Zusicherungen zur Fehlersuche
- Zusicherungen zur Parameterprüfung

```
def years_to_retirement(age, working_years):  
    assert 18 <= age <= 67, 'age must be in range 18 – 67'  
    assert 10 < working_years < 52, 'yeras must be in range 11 – 51'  
  
    return 67 – age
```

```
years_to_retirement(70, 10)
```

- **Schlägt wie folgt fehl:**

AssertionError: age must be in range 18 – 67



PART 7:

Dateiverarbeitung





- Ein wichtiger Bestandteil vieler Anwendungen ist die Verarbeitung von Informationen aus Dateien
 - Python besitzt das Modul `os`, das uns viele nützliche Funktionalitäten zur Verfügung stellt, um mit Verzeichnissen (und auch Dateien) zu arbeiten.
 - Einige weitere Funktionalitäten werden durch das Modul `shutil` bereitgestellt.
 - bei der Kommunikation und Ein- und Ausgabe immer auch zu Fehlern oder Problemen möglich => Exception Handling schon thematisiert
 - Beispiele bieten nur rudimentäre Fehlerbehandlung, um diese kurz zu halten
-



- **Beispiele bieten nur rudimentäre Fehlerbehandlung, um diese kurz zu halten**
- **Beispiele nutzen folgende Verzeichnisstruktur als Ausgangsbasis**

```
files-examples-dir
|-- example-data.csv
|-- example-file.txt
|-- rename-dir
|-- subdir1
`-- subdir2
```



- **Beispiele sollen folgende Verzeichnisstruktur als Ausgangsbasis nutzen:**

```
files-examples-dir
|-- example-data.csv
|-- example-file.txt
|-- rename-dir
|-- subdir1
`-- subdir2
```

- **Wir können ein neues Verzeichnis mit der Funktion `mkdir()` erstellen**

```
>>> import os
>>>
>>> os.mkdir('files-examples-dir')
>>> os.mkdir('files-examples-dir/rename-dir')
>>> os.mkdir('files-examples-dir/subdir1')
>>> os.mkdir('files-examples-dir/subdir2')
```

- **Alternative:**

```
>>> os.makedirs("a/b/c")
```



- **Zum Erzeugen von Dateien gibt es keinen separaten Funktionsaufruf, sondern, es wird über das Öffnen und einen speziellen Modus realisiert.**

```
>>> new_csv_file = open("files-examples-dir/example-data.csv", "x")
```

- **Wir wechseln das aktuelle Verzeichnis mit der Funktion `chdir()`:**

```
>>> os.chdir("files-examples-dir")
```

- **Nun erstellen wir die letzte Datei (ohne Verzeichnisangabe):**

```
>>> new_txt_file = open("example-file.txt", "x")
```

- **Das aktuelle Verzeichnis lässt sich mit `getcwd()` wie folgt abfragen:**

```
>>> print(os.getcwd())
```



- **Das aktuelle Verzeichnis lässt sich mit `listdir()` wie folgt auslesen:**

```
>>> os.listdir()
['example-file.txt', 'rename-dir', 'subdir2', 'example-data.csv', 'subdir1']
```

- **Datei oder Verzeichnis? (`isfile()` / `isdir()`)**

```
>>> dircontent = os.listdir()
>>> for path in dircontent:
...     if os.path.isdir(path):
...         print(path, "is a directory")
...     if os.path.isfile(path):
...         print(path, "is a file")
... 
```



- **Größe mit `getsize()` bestimmen:**

```
>>> os.path.getsize(".")  
384
```

- **Existenzprüfung? (`exists()`)**

```
>>> def check_existence(path):  
...     if os.path.exists(path):  
...         print("File", path, "exists")  
...     else:  
...         print("No such file", path)
```

```
>>> check_existence("example-file.txt")  
File example-file.txt exists  
>>> check_existence("UNKNOWN.UFO")  
No such file UNKNOWN.UFO
```



1. Datei mit `open()` öffnen
2. Aktionen (Lesen und / oder schreiben)
3. Datei mit `close()` schließen

```
file = open("test.txt", encoding = 'utf-8')  
# perform file operations  
file.close()
```

- **Verarbeitungsmodi**

- READ - r
- WRITE – w
- APPEND – a
- CREATE - x

```
file_to_read = open("test.txt")           # Standard 'r'  
file_to_write = open("test.txt", 'w')     # Schreibmodus  
binary_file_rw = open("img.bmp", 'r+b')  # Lesen und Schreiben im Binärmodus
```




1. Datei mit `open()` öffnen
2. Aktionen (Lesen und / oder Schreiben)
3. Datei mit `close()` schließen

```
file = open("test.txt", encoding='utf-8')
try:
    # perform file operations
    pass
finally:
    file.close()
```

- Praktischerweise wird `close()` passend aufgerufen, wir müssen es nicht mehr explizit schreiben – der Aufruf erfolgt intern

```
with open("test.txt", encoding='utf-8') as file:
    # perform file operations
    pass
```



1. Daten schreiben (write())

```
with open("../personenliste.txt", 'w', encoding='utf-8') as file:  
    file.write("Person1: Michael\n")  
    file.write("Person2: Peter\n")  
    file.write("This file contains three lines\n")
```

2. Daten lesen (read())

```
with open("../personenliste.txt", 'r', encoding='utf-8') as file:  
    content = file.read()  
print(content)
```

- Tatsächlich ist das noch leicht unhandlich und erfordert eine Nacharbeit, um an die einzelnen Informationen bzw. Zeilen zu kommen.



1. Daten lesen II (in)

```
with open("../personenliste.txt", 'r', encoding='utf-8') as file:
    # Zeilenweises Lesen
    for line in file:
        print(line, end='')
```

2. Daten lesen III (readlines())

```
with open("../personenliste.txt", 'r', encoding = 'utf-8') as file:
    lines = file.readlines()
    print("Lines", lines)
```



1. Daten lesen IV (readline())

```
with open("../personenliste.txt", 'r', encoding='utf-8') as file:
    count = 1
    line_content = file.readline()
    while line_content:
        print("Line", count, ":", line_content, end = "")
        line_content = file.readline()
        count += 1
```

2. Daten lesen V (readline() mit Walross-Operator := (seit Python 3.8 Assignment-Expr))

```
with open("../personenliste.txt", encoding='utf-8') as file:
    count = 1
    while line_content := file.readline():
        print("Line", count, ":", line_content, end = "")
        count += 1
```



DEMO

`personenliste.py`

Dateiverarbeitung – Kopieren / Umbenennen / Löschen (shutil)



```
import os
import shutil

shutil.copy('personenliste.py', "copy_of_personenliste.py")
print(os.listdir())

os.rename('copy_of_personenliste.py', 'to-be-deleted.py')
print(os.listdir())

os.remove('to-be-deleted.py')
print(os.listdir())

os.mkdir("new_and_empty")
os.rmdir('new_and_empty')
```

Dateiverarbeitung – Löschen von Verzeichnissen (shutil)



```
import os
import shutil
```

```
os.mkdir("dir-to-be-deleted")
open("dir-to-be-deleted/to-be-deleted-1.txt", "x")
os.rmdir('dir-to-be-deleted')
```

OSError: [Errno 66] Directory not empty: 'dir-to-be-deleted'

```
shutil.rmtree("dir-to-be-deleted")
```



JSON-Verarbeitung





1. **JSON steht für JavaScript Object Notation**
 2. **ist ein leichtgewichtiges Format für den Datenaustausch**
 3. **In den letzten 5 bis 10 Jahren war JSON eine der, wenn nicht sogar die beliebteste Art, Daten zu verarbeiten.**
- **Daten in einem dict zu speichern. Dort können weitere verschachtelte Dictionaries und Listen oder andere Typen wie Ganzzahlen und Strings enthalten sein.**
 - **Modul json ermöglicht es ...**
 - dict in einen JSON-String umwandeln – man spricht von Serialisieren.
 - Rückwandlung aus JSON-String in ein dict. Dann spricht man von Deserialisieren
-



- **Daten schreiben wir durch Aufruf von dump():**

```
import json
```

```
data = {"TIM": {"name": "Tim", "age": 50, "city": "Kiel"},  
        "MIKE": {"name": "Mike", "age": 50, "city": "Zürich"},  
        "INFO": ["This", "is", "an", "information"],  
        "CHECKSUM": 4711}
```

```
with open('data.json', 'w') as jsonfile:  
    json.dump(data, jsonfile)
```

- **Daten lesen wir durch Aufruf von load():**

```
with open('data.json') as json_file:  
    data = json.load(json_file)  
    print(data)
```



Exercises Part 7

<https://github.com/Michaeli71/Python-Workshop>





PART 8:

Datumsverarbeitung





- Ein wichtiger Bestandteil vieler Anwendungen ist die Verarbeitung von Informationen aus Datums- und Zeitangaben.
- Python stellt im Modul `datetime` diverse nützliche Funktionalitäten zur Datumsverarbeitung zur Verfügung, deswegen standardmässig:

```
import datetime
```

- Im Vergleich zum Java Date & Time API aber nicht so umfangreich und ausgereift

	Python	Java
Zeitpunkte	<code>datetime</code>	<code>LocalDateTime</code>
Datumsangaben	<code>date</code>	<code>LocalDate</code>
Zeitangaben	<code>time</code>	<code>LocalTime</code>



- **Durch Aufruf der Methode `now()` erhalten wir ein `datetime`-Objekt, das das aktuelle lokale Datum und die Uhrzeit enthält. Zur einfacheren Handhabung erstellen wir eine gleichnamige Hilfsfunktion:**

```
def now():  
    return datetime.datetime.now()
```

- **Wir können ein `datetime`-Objekt basierend auf Angaben zu Jahr, Monat, Tag und optional auch Zeit erzeugen.**

```
# datetime(year, month, day)  
date_no_time_set = datetime.datetime(2020, 11, 23)  
  
# datetime(year, month, day, hour, minute, second, microsecond)  
date_and_time = datetime.datetime(2017, 11, 28, 23, 55, 59, 342380)
```



- **Zuvor haben wir bereits gesehen, dass ein Zeitpunkt aus diversen Bestandteilen besteht. Auf diese können wir per .-Notation und deren Namen zugreifen, etwa auf year, hour und minute:**

```
sunday_afternoon = datetime.datetime(2021, 6, 13, 16, 52, 59)
print("year =", sunday_afternoon.year)
print("month =", sunday_afternoon.month)
print("day =", sunday_afternoon.day)
print("weekday() =", sunday_afternoon.weekday())
print("isoweekday() =", sunday_afternoon.isoweekday())
```

- **=>**

```
year = 2021
month = 6
day = 13
weekday() = 6
isoweekday() = 7
```



- **datetime-Objekte** kann man auch aus einem **Unix-Zeitstempel*** erzeugen.
- Die Methode **fromtimestamp()** wandelt einen Zeitstempel in ein Datum um.
- Eine Rückwandlung geschieht mit **timestamp()**:

```
datetime_from_ts = datetime.datetime.fromtimestamp(43211234)
print(datetime_from_ts.timestamp())
```

- =>

```
43211234.0
```

- **Praktische Funktion:**

```
def seconds_to_datetime(seconds):
    return datetime.datetime.fromtimestamp(seconds)
```

*Das ist die Anzahl der Sekunden zwischen einem bestimmten Datum und dem 1. Januar 1970 in UTC.



- **datetime-Objekte bestehen bekanntlich aus**
 - Datum und
 - Uhrzeit
- **Bislang aber immer in Kombination, nun einzeln betrachten:**

```
sunday_afternoon = datetime.datetime(2021, 6, 13, 16, 52, 59)
print("date() =", sunday_afternoon.date())
print("time() =", sunday_afternoon.time())
```

- **=>**

```
date() = 2021-06-13
time() = 16:52:59
```

- **GLEICH MEHR DAZU ...**
-



- **Zeitdifferenzen ermitteln**

```
start = datetime.datetime(2021, 2, 7, 10, 10, 10)
end = datetime.datetime(2021, 2, 8, 11, 12, 13)
print("end - start =", end - start)
print(type(end - start))
```

- =>

```
end - start = 1 day, 1:02:03
<class 'datetime.timedelta'>
```



- **Zeitdifferenzen ermitteln**

```
td1 = datetime.timedelta(weeks=2, days=3, hours=5, seconds=6)
td2 = datetime.timedelta(days=7, hours=8, minutes=9, seconds=10)
print(td1 - td2)
print(td1 + td2)
```

```
michas_birthday = datetime.date(1971, 2, 7)
print(michas_birthday + td2 * 2)
```

- =>

```
9 days, 20:50:56
24 days, 13:09:16
1971-02-21
```



- Die Klasse `date` besitzt eine `today()`-Methode. Um den Zugriff innerhalb etwas einfacher zu gestalten, erstellen wir eine gleichnamige Hilfsfunktion:

```
def today():  
    return datetime.date.today()
```

- Wir können ein `date`-Objekt basierend auf Angaben zu Jahr, Monat, Tag erzeugen oder basierend auf einem ISO-String:

```
# date(year, month, day)  
sophies_birthday = datetime.date(2020, 11, 23)  
  
date_from_iso = datetime.date.fromisoformat('2020-11-23')
```



- Ein Zeitpunkt besteht aus diversen Bestandteilen. Auf diese können wir per **.-Notation** und deren Namen zugreifen, etwa auf **year**, **month** und **day**:

```
date_from_iso = datetime.date.fromisoformat('2020-11-23')
print("Current year:", date_from_iso.year)
print("Current month:", date_from_iso.month)
print("Current day:", date_from_iso.day)
print("weekday():", date_from_iso.weekday())
print("isoweekday():", date_from_iso.isoweekday())
```

- =>

```
Current year: 2020
Current month: 11
Current day: 23
weekday(): 0
isoweekday(): 1
```



- **Um den Wochentag zu einem Datum zu ermitteln, schreiben wir uns folgende Hilfsfunktion, die auf `strftime()` basiert**

```
def get_week_day(date):  
    return date.strftime('%A')
```

- **Tag im Monat**

```
def day_of_month(date):  
    return date.day
```

- **Tag im Jahr**

```
def day_of_year(date):  
    return date.timetuple().tm_yday
```

```
def day_of_year_2(date):  
    return date.strftime('%j')
```



- **Länge eines Monats über `timedelta.days` bestimmen:**

```
print((datetime.date(2012, 3, 1) - datetime.date(2012, 2, 1)).days)
print((datetime.date(2014, 3, 1) - datetime.date(2014, 2, 1)).days)
```

- **Allgemeingültige Hilfsfunktion:**

```
def length_of_month(month, year):
    start = datetime.date(year, month, 1)
    month += 1
    if month > 12:
        year += 1
        month = 1

    end = datetime.date(year, month, 1)
    return (end - start).days
```



- **Länge eines Jahres über `timedelta.days` bestimmen. Allgemeingültige Hilfsfunktion:**

```
def length_of_year(year):  
    start = datetime.date(year, 1, 1)  
    end = datetime.date(year + 1, 1, 1)  
    return (end - start).days
```

- **Schaltjahr bestimmen: Es ist wünschenswert, programmatisch ermitteln zu können, ob ein Jahr ein Schaltjahr ist. Dabei hilft uns das Modul `calendar` und die Methode `isleap()`.**

```
import calendar
```

```
print("2012 is leap?", calendar.isleap(2012))
```



- **Datum aus Zeitstempel erzeugen**

```
date_from_ts = datetime.date.fromtimestamp(34774300)  
print(date_from_ts)
```

- **=> 1971-02-07**

- **Manchmal ist es hilfreich, die Tage seit Christi Geburt fortlaufend als Zahlenwert zu zählen. `toordinal()` wandelt ein Datum in einen Zahlenwert, der der Anzahl an Tagen seit dem 1. Januar im Jahr 1 entspricht:**

```
christmas_eve_20 = datetime.date.fromisoformat('2020-12-24')  
print(christmas_eve_20.toordinal())
```

- **=> 737783**



- Die Klasse `datetime` besitzt die Methoden `now()` und `time()`, Um den Zugriff auf die aktuelle Uhrzeit etwas einfacher zu gestalten, erstellen wir eine Hilfsfunktion:

```
def current_time():  
    return datetime.datetime.now().time()
```

- Wir können ein `time` basierend auf Angaben zu Stunde, Minute und Sekunde erzeugen oder optional noch Millisekunde:

```
# time(hour, minute, second)  
print(datetime.time(11, 28, 45))
```

```
# time(hour, minute, second, microsecond)  
print(datetime.time(11, 28, 45, 6789))
```



DEMO

`datetime_intro.py`



DEMO

`calendar_printer.py`



Exercises Part 8

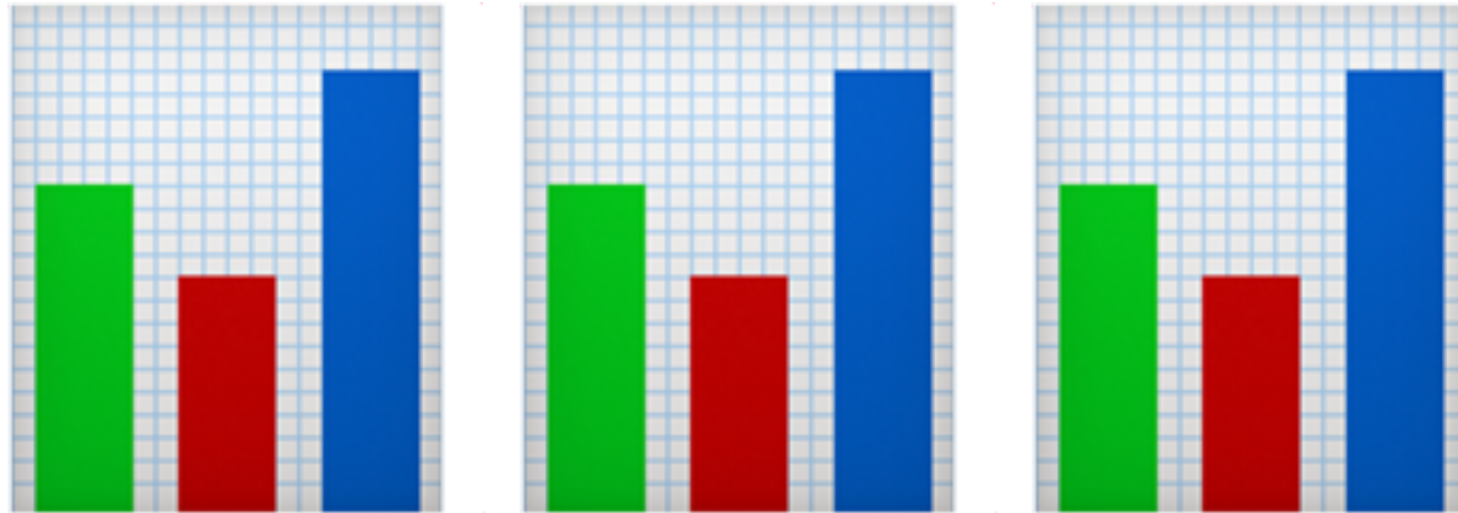
<https://github.com/Michaeli71/Python-Workshop>





Python Quick Start

Projects & Productivity Showdown





PART 99: BONUS





«Reflection»





- Als Ausgangsbasis dient folgende Klasse mit ein paar Attributen

```
class MyClass:
    def __init__(self):
        self.attr = 7271
        self.info = "This is PUBLIC"
        self.__private_info = "My birthday"

    def get_private_info(self):
        return self.__private_info

obj = MyClass()
```

- Ein Attribut ist «privat» und bietet nur lesenden Zugriff per Methode `get_private_info()`



- **Prüfe auf Existenz (hasattr())**

```
print("hasattr()")
print(hasattr(obj, "__init__"))
print(hasattr(obj, "info"))
print(hasattr(obj, "__private_info"))
print(hasattr(obj, "_MyClass__private_info"))
print(hasattr(obj, "unknown"))
```



- =>

```
hasattr()
True
True
False
True
False
```





- Zugriff auf Werte mit Fallback (getattr())

```
print("getattr()")
print(getattr(obj, "__init__"))
print(getattr(obj, "attr"))
print(getattr(obj, "info"))
print(getattr(obj, "unknown", "FALLBACK"))
```

- =>

```
getattr()
<bound method MyClass.__init__ of <__main__.MyClass object at 0x101009fd0>>
7271
This is PUBLIC
FALLBACK
```



- **Prüfung auf Ausführbarkeit (callable())**

```
print("callable()")  
print(callable(getattr(obj, "__init__")))  
print(callable(getattr(obj, "get_private_info")))  
print(callable(getattr(obj, "attr")))
```

- =>

```
callable()  
True  
True  
False
```



- **Modifikation von Werten (setattr())**

```
print("setattr()")
setattr(obj, "attr", 1234)
setattr(obj, "info", "PIN-CODE")
setattr(obj, "_MyClass__private_info", "!!UNEXPECTED!!")
print(obj.attr)
print(obj.info)
print(obj._MyClass__private_info)
```

- =>

```
setattr()
1234
PIN-CODE
!!UNEXPECTED!!
```



«HTTP-Support»





- **Installation**

```
pip3 install requests
```

- **Zugriff auf die Python Home Page**

```
import requests
```

```
def print_response_info(address):  
    response = requests.get(address)  
    # print(response)  
    print(response.status_code)  
    print(response.headers)  
    print(response.text[0:100])  
    print(response.content[0:100])
```

```
print_response_info("https://www.python.org/")
```



- **Daten als Datei sichern**

```
import requests
```

```
def print_response_info(address):  
    response = requests.get(address)  
    print(response.text)  
    print(response.content)
```

```
with open("../ch99_web_http/python.html", "w") as html_file:  
    html_file.write(response.text)
```

```
print_response_info("https://www.python.org/")
```



- **Daten als Datei sichern**

```
import requests
```

```
response = requests.get("https://imgs.xkcd.com/comics/modern_tools.png")  
print(response)  
print(response.status_code)
```

```
with open("../ch99_web_http/modern_tools.png", "wb") as png_file:  
    png_file.write(response.content)
```



- **Wetterdaten auslesen**

```
import requests
```

```
# https://openweathermap.org/api
```

```
api_key = "74e5520ecf1361c1de73dec5cc4a9c9"
```

```
city = input("Bitte geben Sie den Städtenamen für die Wetterabfrage ein: ")
```

```
url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={api_key}"
```

```
data = requests.get(url).json()
```

```
print(data)
```

```
temp = data["main"]["temp"]
```

```
print("Temperatur in °F:", temp)
```

```
humidity = data["main"]["humidity"]
```

```
print("Feuchtigkeit:", humidity)
```



DEMO

`second_requests.py`
`third_requests_post.py`
`image_download.py`
`weather_api.py`



«HTTP/2-Support»





- **Installation**

```
pip3 install httpx    =>. pip3 install 'httpx[http2]'
```

- **Zugriff auf die Python Home Page**

```
import httpx
```

```
def print_response_info(address):  
    client = httpx.Client(http2=True)  
    response = client.get(address)  
    print(response.status_code)  
    print(response.http_version)  
    print(response.headers)  
    print(response.text[0:100])  
    print(response.content[0:100])
```

```
200  
HTTP/2  
Headers({'content-type': 'text/html;  
charset=utf-8', 'via': '1.1 varnish,  
1.1 varnis...
```

```
print_response_info("https://www.python.org/")
```



«Testing mit Pytest»





- Für Tests sofort (nach Installation) einsetzbar (<https://docs.pytest.org/en/stable/>)
- Testbehauptungen werden einfach mit dem Schlüsselwort `assert` formuliert

```
def test_answer():  
    assert sum([1, 2, 3]) == 6
```

- Auf einfache Weise auch parametrisierte Tests möglich

```
import pytest  
  
@pytest.mark.parametrize("test_input,expected",  
                          [ ("2 + 5", 7), ("2 ** 4", 16), ("6 * 9", 42)])  
def test_eval(test_input, expected):  
    assert eval(test_input) == expected
```

Pytest Quick Start (RECAP)



- Gute Integration in PyCharm

```
import pytest
```

```
def test_answer():  
    assert sum([1, 2, 3]) == 6
```

```
@pytest.mark.parametrize("test_input,expected",  
                           [ ("2 + 5", 7), ("2 ** 4", 16), ("6 * 9", 42)])  
def test_eval(test_input, expected):  
    assert eval(test_input) == expected
```

Test Results	0 ms
test_ex02_repeat_chars	0 ms
test_repeat_chars_ABCD	0 ms
test_repeat_chars_ABCDEF	0 ms
test_answer	0 ms
test_eval	0 ms
(2 + 5-7)	0 ms
(2 ** 4-16)	0 ms
(6 * 9-42)	0 ms



Wohlgeformte Klammern





Schreiben Sie eine Methode `checkBraces()` bzw. Funktion `check_braces()`, die prüft, ob die als String gegebene Folge von runden Klammern jeweils passende (sauber geschachtelte) Klammerpaare enthält.

Eingabe	Resultat	Kommentar
"()"	True	
"()()"	True	
"(())(())"	False	zwar gleich viele öffnende und schließende Klammern, aber nicht sauber geschachtelt
"((("	False	keine passende Klammerung



```
boolean checkBraces(final String input)
{
    int openingCount = 0;

    for (char ch : input.toCharArray())
    {
        if (ch == '(')
        {
            openingCount++;
        }
        else if (ch == ')')
        {
            openingCount--;
            if (openingCount < 0)
                return false;
        }
    }

    return openingCount == 0;
}
```

```
def check_braces(input):
    opening_count = 0

    for ch in input:
        if ch == "(":
            opening_count += 1
        elif ch == ")":
            opening_count -= 1
            if opening_count < 0:
                return False

    return opening_count == 0
```



```
@ParameterizedTest(name="checkBraces('{0}') -- Hinweis: {2}" )
@CsvSource({ "()", true, ok",
             "()", true, ok",
             "())(()", false, nicht sauber geschachtelt",
             "()", false, keine passende Klammerung",
             ")()", false, startet mit schliessender Klammer" })
```

```
void checkBraces(String input, boolean expected, String hint)
{
    boolean result = Ex09_SimpleBracesChecker.checkBraces(input);

    assertEquals(expected, result);
}
```

```
@pytest.mark.parametrize("input, expected, hint",
                          [
                              ("()", True, "ok"),
                              "()", True, "ok"),
                              ("())(()", False, "nicht sauber geschachtelt"),
                              "()", False, "keine passende Klammerung"),
                              ")()", False, "startet mit schliessender Klammer")])
```

```
def test_check_braces(input, expected, hint):
    assert check_braces(input) == expected
```





«QR Code»





- **Zunächst das Modul treepoem installieren:**

```
pip install treepoem
```

- **QR Code generieren**

```
import treepoem
```

```
img = treepoem.generate_barcode(  
    barcode_type='qrcode',  
    data='https://github.com/Michaeli71/Python-For-Machine-Learning-2025/',  
    options={"eclevel": "Q"}  
)  
img.convert('1').save('Python-For-Machine-Learning-2025.png')
```



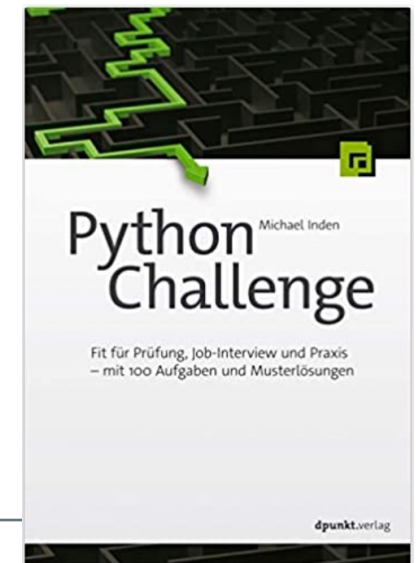
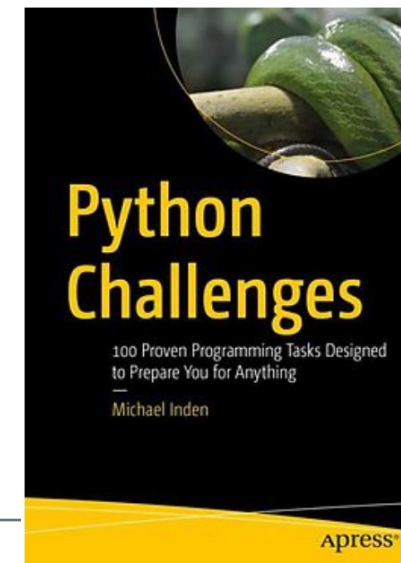
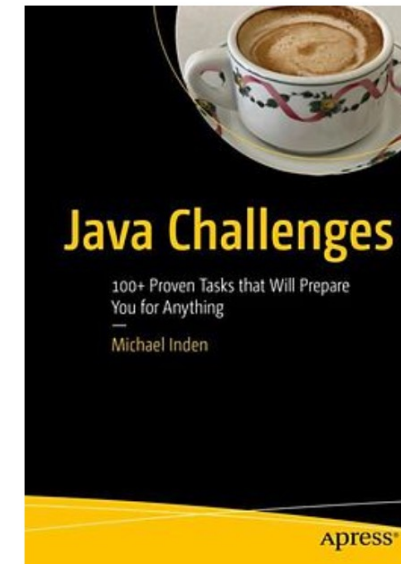
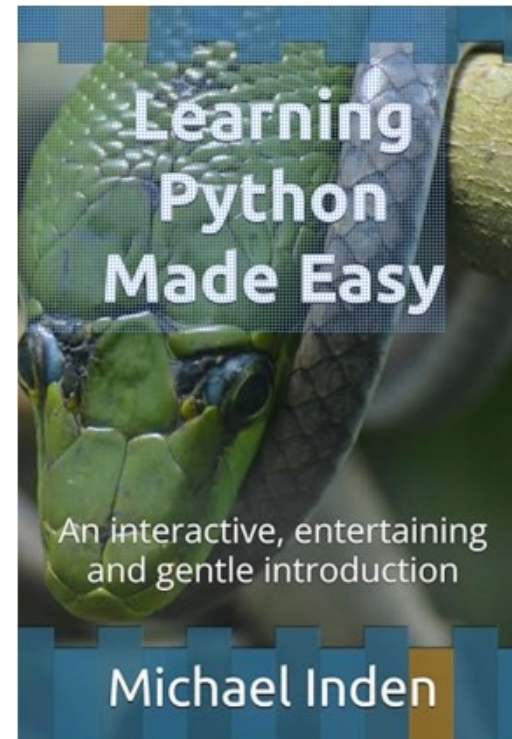
DEMO

`qr-generator.py`



Questions?

Help





Thank You
